Лабораторна робота № 3

Діагностика шкал вимірювання

Курс СК26 «Інтелектуальний аналіз та візуалізація даних»

Тривалість: 2 год аудиторно + 3 год самостійно · **Оцінка:** 4 бали **Файли:**

`Ковід_дані_навчальні.xlsx` , `ЛР3_помилковий_аналіз.ipynb`

Шкала вимірювання визначає, які операції над змінною взагалі мають сенс. Комп'ютер цього не знає: він порахує середнє від номера телефону й не поскаржиться. Знати має вимірювач.

У цій роботі ви складаєте **паспорт восьми змінних** свого варіанта, показуєте на власних даних, які статистики для них недопустимі, і виправляєте чужий аналіз, де таких помилок десять.

In [1]:
```
FULL_NAME = 'Цимбалюк Антон'      # напишіть тут своє прізвище та ім'я
GROUP     = 'ШІ-34'
NUMBER    = 15      # ваш номер за списком групи (1–30)

assert FULL_NAME and GROUP and NUMBER, 'Заповніть три поля вище'
assert 1 <= NUMBER <= 30, 'Номер має бути від 1 до 30'
```

In [2]:
```
import json
import re
import math
import textwrap
import datetime as dt
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_colwidth', 60)

VARIANTS = json.load(open('lr3_variants_student.json'))
VARIANT = VARIANTS[NUMBER - 1]

print('=' * 74)
print(f'ВАРІАНТ {VARIANT["варіант"]}   ·   {FULL_NAME}   ·   {GROUP}')
print('=' * 74)
print()
print('Ваші вісім стовпців:')
for i, c in enumerate(VARIANT['стовпці'], 1):
    print(f'   {i}. {c}')
print()
print('Ваша пара для аналізу зв’язку:', ' та '.join(VARIANT['пара']))
print()
print('Ваше додаткове питання:')
print('  ', VARIANT['питання'])
```

```
================================================================================
ВАРІАНТ 15   ·   Цимбалюк Антон   ·   ШІ-34
================================================================================

Ваші вісім стовпців:
   1. Телефон
   2. Стать
   3. ступінь КТ
   4. Виписка
   5. КТ
   6. Т вище 37,5
   7. Сечов2
   8. БРпр1

Ваша пара для аналізу зв’язку: Стать та ступінь КТ

Ваше додаткове питання:
   Чому не можна обчислити коефіцієнт кореляції Пірсона між статтю та результатом лікування? Яку міру зв’язку слід у
зяти замість нього?
```

In [3]:
```
DATA_FILE = 'Ковід_дані_навчальні.xlsx'
df = pd.read_excel(DATA_FILE, sheet_name='Лист1', engine='openpyxl')
MY_COLS = VARIANT['стовпці']

print('Розмір набору:', df.shape)
df[MY_COLS].head(8)
```

```
Розмір набору: (121, 77)
```

Out[3]:

	Телефон	Стать	ступінь КТ	Виписка	КТ	Т вище 37,5	Сечов2	БРпр1
0	0958469788 дружина Ганна	ч	сердній	2022-03-10 00:00:00	20-30%	3	6.5	3.4
1	0671778486, 0994887663 дружина Оксана	ч	середній	2022-03-16 00:00:00	30-35%	3	4	2.8
2	0958768045, чоловік Степан 0951784726	ж	критичний	2022-01-18 00:00:00	70-80%	3	10.1	2.8
3	0959819331. дочка Валентина 0960845205	ж	критичний	2022-01-13 00:00:00	0.8	6	12.5	3.4
4	0952294866, дочка Дарина 0678360161	ж	NaN	2022-03-11 00:00:00	NaN	5 до гос	7.4	3.4
5	0959922983 сестра Софія	ж	середній	2021-12-26 00:00:00	30-40%	NaN	21.4	3.4
6	0998073702, дружина Наталія 0998382226	ч	важкий	2022-05-11 00:00:00	60-65%	NaN	6.1	5.1
7	0637258771, син Сергій 0968413301	ж	легкий	2022-05-28 00:00:00	до 17%	NaN	7.6	5.1

Етап 1. Паспорт змінних

Для кожного з восьми стовпців визначте шкалу вимірювання. Спирайтеся не на тип даних у pandas, а на **зміст величини**: що означають її значення і які порівняння між ними мають сенс.

Нагадування:

Шкала	Ознака
Номінальна	значення можна лише порівняти на рівність
Порядкова	є природний порядок, але відстані між сусідніми рівнями невідомі
Інтервальна	різниці мають сенс, нуль умовний
Відношень	є природний нуль, відношення мають сенс
Абсолютна	природні і нуль, і одиниця вимірювання

Окремо позначайте **ідентифікатори** — значення, що не є величиною взагалі (номер історії хвороби, прізвище, телефон).

In [4]:
```
# Спершу подивіться, що каже pandas — і не довіряйте йому
df[MY_COLS].info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 121 entries, 0 to 120
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Телефон      120 non-null    object
 1   Стать        121 non-null    str
 2   ступінь КТ   42 non-null     str
 3   Виписка      120 non-null    object
 4   КТ           44 non-null     object
 5   Т вище 37,5  78 non-null     object
 6   Сечов2       102 non-null    object
 7   БРпр1        109 non-null    float64
dtypes: float64(1), object(5), str(2)
memory usage: 7.7+ KB
```

In [5]:

```python
# Далі — що є всередині насправді
for c in MY_COLS:
    vals = df[c].dropna()
    types_str = ', '.join(f'{t.__name__}×{n}' for t, n in vals.map(type).value_counts().items())
    print(f'--- {c}')
    print(f'    непорожніх {len(vals)} з {len(df)} | унікальних {vals.nunique()} | типи: {types_str}')
    print(f'    приклади: {[repr(v) for v in vals.unique()[:4]]}')
    print()
```

```
--- Телефон
    непорожніх 120 з 121 | унікальних 120 | типи: str×97, int×23
    приклади: ["'0958469788 дружина Ганна'", "'0671778486, 0994887663 дружина Оксана'", "'0958768045, чоловік Степан
0951784726'", "'0959819331. дочка Валентина 0960845205'"]

--- Стать
    непорожніх 121 з 121 | унікальних 4 | типи: str×121
    приклади: ["'ч'", "'ж'", "'чол'", "'жін'"]

--- ступінь КТ
    непорожніх 42 з 121 | унікальних 12 | типи: str×42
    приклади: ["'сердній'", "'середній'", "'критичний'", "'важкий'"]

--- Виписка
    непорожніх 120 з 121 | унікальних 90 | типи: datetime×118, str×2
    приклади: ['datetime.datetime(2022, 3, 10, 0, 0)', 'datetime.datetime(2022, 3, 16, 0, 0)', 'datetime.datetime(20
22, 1, 18, 0, 0)', 'datetime.datetime(2022, 1, 13, 0, 0)']

--- КТ
    непорожніх 44 з 121 | унікальних 26 | типи: float×27, str×17
    приклади: ["'20-30%'", "'30-35%'", "'70-80%'", '0.8']

--- Т вище 37,5
    непорожніх 78 з 121 | унікальних 35 | типи: str×48, int×30
    приклади: ['3', '6', "'5 до гос'", "'3 дні в стац'"]

--- Сечов2
    непорожніх 102 з 121 | унікальних 66 | типи: float×79, int×22, str×1
    приклади: ['6.5', '4', '10.1', '12.5']

--- БРпр1
    непорожніх 109 з 121 | унікальних 20 | типи: float×109
    приклади: ['np.float64(3.4)', 'np.float64(2.8)', 'np.float64(5.1)', 'np.float64(4.5)']
```

Що видно вже з цього виводу

`info()` називає шість із восьми стовпців `object` — і це не шкала, а відмова pandas щось стверджувати. Усередині чотирьох стовпців лежать **два різні кодування одночасно**:

- `Телефон` — 97 рядків («`0958469788 дружина Ганна`») і 23 цілих числа;
- `Виписка` — 118 дат і 2 рядки з неіснуючими місяцями (`'24.024.2021'`);
- `КТ` — 27 часток (`0.3`) і 17 текстових клітинок: 16 діапазонів (`'30-40%'`) і один рядок із самих пробілів;
- `Т вище 37,5` — 30 чисел і 48 фраз («`5 днів до госп`», «`3 дні в стац`»).

Тип `object` тут означає рівно одне: **Excel прийняв усе, що набрав лікар**. Шкалу доведеться визначати за змістом величини, а рахувати — лише після того, як обидва кодування зведено до одного.

In [6]:
```python
# Скільки різних написань ховається за кожною «категорією»
for c in ['Стать', 'ступінь КТ']:
    print(f'--- {c}: {df[c].nunique()} різних написань')
    print(df[c].value_counts().to_string())
    print()
```

```
--- Стать: 4 різних написань
Стать
ж      56
ч      34
жін    19
чол    12

--- ступінь КТ: 12 різних написань
ступінь КТ
середній         10
важкий            9
серед ступ        6
легкий            5
критичний         3
вкрай важкий      3
сердній           1
легка             1
легкий            1
вкрай важкий      1
важкий            1
серед ступеня     1
```

Заповніть паспорт

Для кожного стовпця вкажіть шкалу та **обґрунтування** — одне речення про те, чому саме ця шкала. Обґрунтування оцінюється, назва шкали сама по собі — ні.

In [7]:
```python
PASSPORT = {
    # 'назва стовпця': ('шкала', 'обґрунтування однією фразою'),

    'Телефон': ('ідентифікатор',
        'контактний номер — мітка для зв'язку, а не величина: 120 унікальних значень на '
        '120 заповнених клітинок, і єдина осмислена операція над ними — порівняння на рівність'),

    'Стать': ('номінальна',
        'дві категорії без будь-якого порядку — «ч» не більше і не менше за «ж», тож значення '
        'можна лише лічити за групами; у файлі ці дві категорії записані чотирма написаннями'),

    'ступінь КТ': ('порядкова',
        'рівні легкий < середній < важкий < вкрай важкий < критичний мають природний порядок, '
        'але відстані між сусідніми рівнями невідомі й нерівні — це вердикт рентгенолога, не вимір'),

    'Виписка': ('інтервальна',
        'дата: різниця двох дат — тривалість у днях і має зміст, але нуль календаря умовний, '
        'тому фраза «удвічі пізніша дата» не означає нічого'),

    'КТ': ('абсолютна',
        'частка ураженої тканини легень: природні і нуль (ураження немає), і одиниця (уражено все), '
        'величина безрозмірна, іншої одиниці вимірювання для неї не існує'),

    'Т вище 37,5': ('відношень',
        'кількість днів гарячки: нуль природний (днів гарячки немає) і 6 днів справді вдвічі довше '
        'за 3, але сама одиниця — доба — угода, її можна замінити на години'),

    'Сечов2': ('відношень',
        'концентрація сечовини в крові: нульова концентрація — природний нуль, тому 14 ммоль/л '
        'справді вдвічі більше за 7, а одиниця вимірювання (ммоль/л) обрана за домовленістю'),

    'БРпр1': ('відношень',
        'прямий білірубін: нульова концентрація фізично означає відсутність речовини, тож відношення '
        'мають зміст, а вибір одиниці (мкмоль/л) умовний і на шкалу не впливає'),
}

for c in MY_COLS:
    PASSPORT.setdefault(c, ('', ''))

passport_df = pd.DataFrame(
    [(c, PASSPORT[c][0], PASSPORT[c][1]) for c in MY_COLS],
    columns=['Стовпець', 'Шкала', 'Обґрунтування'])
passport_df
```

Out[7]:

	Стовпець	Шкала	Обґрунтування
0	Телефон	ідентифікатор	контактний номер — мітка для зв'язку, а не величина: 120...
1	Стать	номінальна	дві категорії без будь-якого порядку — «ч» не більше і н...
2	ступінь КТ	порядкова	рівні легкий < середній < важкий < вкрай важкий < критич...
3	Виписка	інтервальна	дата: різниця двох дат — тривалість у днях і має зміст, ...
4	КТ	абсолютна	частка ураженої тканини легень: природні і нуль (ураженн...
5	Т вище 37,5	відношень	кількість днів гарячки: нуль природний (днів гарячки нем...
6	Сечов2	відношень	концентрація сечовини в крові: нульова концентрація — пр...
7	БРпр1	відношень	прямий білірубін: нульова концентрація фізично означає в...

Два місця, де я відступив від «очевидної» відповіді

`КТ` **— абсолютна, а не відношень.** Різниця між цими двома шкалами не в нулі (він природний в обох), а в **одиниці вимірювання**. Ммоль/л, доба, міліметр ртутного стовпа — усе це угоди, їх можна замінити іншими. Частка ураженої легені вибору одиниці не має: ціла легеня — це 1, і нічого іншого за одиницю взяти не можна. Саме так методичка визначає абсолютну шкалу («природні і нуль, і одиниця»), і частка — її канонічний приклад.

`Т вище 37,5` **— відношень, а не абсолютна**, попри те що це теж лічба. Одиницею тут є доба, і вона умовна: ту саму тривалість гарячки можна записати в годинах, і всі числа зміняться. Нуль при цьому природний — тому відношень, а не інтервальна.

In [8]:
```
# --- САМОПЕРЕВІРКА 1 ---
ALLOWED_SCALES = {'номінальна', 'порядкова', 'інтервальна', 'відношень',
                  'абсолютна', 'ідентифікатор'}
for c in MY_COLS:
    scale_name, reason = PASSPORT[c]
    assert scale_name.strip().lower() in ALLOWED_SCALES, f'{c}: шкалу «{scale_name}» не розпізнано'
    assert len(reason.strip()) >= 25, f'{c}: обґрунтування закоротке'
print('Самоперевірка 1 пройдена: усі вісім стовпців мають шкалу та обґрунтування.')
```

```
Самоперевірка 1 пройдена: усі вісім стовпців мають шкалу та обґрунтування.
```

Етап 2. Допустимі статистики

Для кожного стовпця обчисліть **тільки те, що дозволяє його шкала**.

- номінальна та ідентифікатор — мода, частоти
- порядкова — додатково медіана та квантилі
- інтервальна — додатково середнє та стандартне відхилення
- відношень і абсолютна — усе, зокрема коефіцієнт варіації

Якщо для стовпця не можна порахувати нічого, крім частот, — так і напишіть.

Спершу — шар зведення кодувань

Шкала каже, **яку** статистику можна брати. Але взяти її можна лише з величин, зведених до одного кодування: інакше `mean()` порахує середнє не по стовпцю, а по тій його частині, яку випадково зрозумів `pd.to_numeric`.

Це **не прибирання заради охайності**: кожна функція нижче тримається за шкалу свого стовпця. Для `ступінь КТ` я задаю порядок рівнів вручну — саме тому, що шкала порядкова, а алфавіт цього порядку не знає.

In [9]:

```
CT_ORDER = ['легкий', 'середній', 'важкий', 'вкрай важкий', 'критичний']
ORDERS = {'ступінь КТ': CT_ORDER}


def clean_sex(v):
    # ч / чол -> 'ч';  ж / жін -> 'ж'. Перша літера розрізняє дві категорії однозначно.
    if pd.isna(v):
        return np.nan
    return {'ч': 'ч', 'ж': 'ж'}.get(str(v).strip().lower()[:1], np.nan)


def clean_degree(v):
    # 12 написань -> 5 рівнів. Порядок рівнів задано в CT_ORDER, а не алфавітом.
    if pd.isna(v):
        return np.nan
    t = str(v).strip().lower()
    if t.startswith('вкрай'):
        return 'вкрай важкий'
    if t.startswith('критич'):
        return 'критичний'
    if t.startswith('важк'):
        return 'важкий'
    if t.startswith('легк'):
        return 'легкий'
    if t.startswith('серед') or t.startswith('сердн'):   # «сердній» — друкарська помилка
        return 'середній'
    return np.nan


def clean_ct(v):
    # Два кодування в одному стовпці: частка (0.3) і текстовий діапазон ('30-40%').
    # Зводжу обидва до відсотків. Діапазон замінюю серединою — це втрата точності,
    # але вона набагато менша, ніж викинути значення зовсім.
    if pd.isna(v):
        return np.nan
    if isinstance(v, (int, float, np.integer, np.floating)):
        return float(v) * 100
    t = str(v).strip().replace('%', '')
    nums = [float(x) for x in re.findall(r'\d+(?:[.,]\d+)?', t.replace(',', '.'))]
    return sum(nums) / len(nums) if nums else np.nan


def fever_days(v):
    # Кількість днів із вільного тексту: '5 днів до госп' -> 5.0
    if pd.isna(v):
        return np.nan
    if isinstance(v, (int, float, np.integer, np.floating)):
        return float(v)
    m = re.search(r'\d+', str(v))
    return float(m.group()) if m else np.nan


def fever_period(v):
    # До якого періоду належать ці дні. Без цієї ознаки самі числа непорівнювані.
    if pd.isna(v):
        return np.nan
    if isinstance(v, (int, float, np.integer, np.floating)):
        return 'не вказано'
    t = str(v).lower()
    in_hospital = ('стац' in t) or ('реан' in t)
    before_hospital = ('гос' in t) or ('вдома' in t)
    if in_hospital and before_hospital:
        return 'змішаний'
    if in_hospital:
        return 'у стаціонарі'
    if before_hospital:
        return 'до госпіталізації'
    return 'не вказано'


CLEAN = {
    'Телефон':     df['Телефон'].astype(str).str.strip(),      # лишається міткою
    'Стать':       df['Стать'].map(clean_sex),
    'ступінь КТ':  df['ступінь КТ'].map(clean_degree),
    'Виписка':     pd.to_datetime(df['Виписка'], errors='coerce'),
    'КТ':          df['КТ'].map(clean_ct),
    'Т вище 37,5': df['Т вище 37,5'].map(fever_days),
    'Сечов2':      pd.to_numeric(df['Сечов2'], errors='coerce'),
    'БРпр1':       pd.to_numeric(df['БРпр1'], errors='coerce'),
}
FEVER_PERIOD = df['Т вище 37,5'].map(fever_period)

print('КАТЕГОРІЙНІ СТОВПЦІ: написань -> справжніх категорій')
```

```
print(f'{"стовпець":<14}{"написань":>10}{"категорій":>11}')
print('-' * 35)
for c in ['Стать', 'ступінь КТ']:
    print(f'{c:<14}{df[c].nunique():>10}{CLEAN[c].nunique():>11}')
print()

print('СТОВПЦІ З ВЕЛИЧИНАМИ: скільки значень розбирає наївний парсер і скільки — наш')
print(f'{"стовпець":<14}{"непорожніх":>12}{"наївно":>8}{"після зведення":>17}{"різниця":>9}')
print('-' * 60)
for c in ['КТ', 'Т вище 37,5', 'Сечов2', 'БРпр1', 'Виписка']:
    naive_n = (df[c].map(lambda v: isinstance(v, (pd.Timestamp, dt.datetime))).sum() if c == 'Виписка'
               else pd.to_numeric(df[c], errors='coerce').notna().sum())
    clean_n = CLEAN[c].notna().sum()
    print(f'{c:<14}{int(df[c].notna().sum()):>12}{int(naive_n):>8}{int(clean_n):>17}{int(clean_n -
naive_n):>+9}')
print()
print('Сечов2 і БРпр1 нічого не виграли: там pd.to_numeric і є зведенням (у Сечов2 воно')
print('правильно відкидає єдиний рядок «немає»). Виписка теж не виграла нічого — і це')
print('важливіше: два записи «24.024.2021» і «20.24.2021» не рятує жоден парсер, бо')
print('24-го місяця не існує, і відновити справжню дату з них неможливо.')
print()
print('Телефон у цю таблицю не входить: він ідентифікатор, і «розбирати» його не треба')
print('взагалі — у CLEAN він лишається рядком. 23 клітинки зі 120 випадково схожі')
print('на число лише тому, що лікар не дописав туди імені родича.')
```

```
КАТЕГОРІЙНІ СТОВПЦІ: написань -> справжніх категорій
стовпець        написань  категорій
-----------------------------------
Стать                  4          2
ступінь КТ            12          5

СТОВПЦІ З ВЕЛИЧИНАМИ: скільки значень розбирає наївний парсер і скільки — наш
стовпець        непорожніх  наївно   після зведення  різниця
------------------------------------------------------------
КТ                      44      27               43      +16
Т вище 37,5             78      30               78      +48
Сечов2                 102     101              101       +0
БРпр1                  109     109              109       +0
Виписка                120     118              118       +0

Сечов2 і БРпр1 нічого не виграли: там pd.to_numeric і є зведенням (у Сечов2 воно
правильно відкидає єдиний рядок «немає»). Виписка теж не виграла нічого — і це
важливіше: два записи «24.024.2021» і «20.24.2021» не рятує жоден парсер, бо
24-го місяця не існує, і відновити справжню дату з них неможливо.

Телефон у цю таблицю не входить: він ідентифікатор, і «розбирати» його не треба
взагалі — у CLEAN він лишається рядком. 23 клітинки зі 120 випадково схожі
на число лише тому, що лікар не дописав туди імені родича.
```

In [10]:

```
def allowed_stats(column, scale):
    # Повертає словник статистик, дозволених для цієї шкали.
    # Рахує не за сирим стовпцем, а за CLEAN[стовпець]: шкала визначає, ЯКУ статистику
    # можна брати, але взяти її можна лише з величин в одному кодуванні.
    # Кожна гілка завершується return: шкали вкладені одна в одну, і нижча не має
    # діставати того, що дозволено лише вищій.
    vals = CLEAN[column].dropna()
    stats = {'непорожніх': len(vals), 'унікальних': vals.nunique()}
    scale_name = scale.strip().lower()

    # — ідентифікатор: мода й частоти формально рахуються, але нічого не кажуть
    if scale_name == 'ідентифікатор':
        stats['частка унікальних'] = round(vals.nunique() / len(vals), 3)
        stats['найбільша частота'] = int(vals.value_counts().max())
        stats['можна'] = 'порівняння на рівність і пошук дублікатів'
        stats['про моду'] = ('формально рахується, але беззмістовна: найчастіше значення '
                             f'трапляється {int(vals.value_counts().max())} раз(и)')
        stats['не можна'] = 'медіана, середнє, будь-яка арифметика над значеннями'
        return stats

    # — номінальна і вище: мода й частоти
    mode = vals.mode()
    stats['мода'] = mode.iloc[0] if len(mode) else None
    if vals.nunique() <= 12:
        stats['частоти'] = vals.value_counts().to_dict()

    if scale_name == 'номінальна':
        stats['не можна'] = 'медіана й середнє — порядку між категоріями немає'
        return stats

    # — порядкова: медіана й квантилі, але через заданий порядок рівнів
    if scale_name == 'порядкова':
        levels = ORDERS[column]
        codes = pd.Categorical(vals, categories=levels, ordered=True).codes
        codes = pd.Series(codes[codes >= 0])
```

```
        stats['медіана'] = levels[int(np.floor(codes.median()))]
        stats['квартилі'] = [levels[int(np.floor(q))] for q in codes.quantile([0.25, 0.75])]
        stats['не можна'] = 'середнє й СКВ — відстані між сусідніми рівнями невідомі'
        return stats

    # — інтервальна: середнє й розсіяння є, але нуль шкали умовний
    if scale_name == 'інтервальна':
        stats['медіана'] = vals.median()
        stats['середнє'] = vals.mean()
        days_from_start = (vals - vals.min()).dt.days
        stats['ст. відхилення, днів'] = round(float(days_from_start.std()), 2)
        stats['розмах, днів'] = int(days_from_start.max())
        stats['не можна'] = 'коефіцієнт варіації й відношення — нуль календаря умовний'
        return stats

    # — відношень і абсолютна: усе, зокрема коефіцієнт варіації
    stats['медіана'] = round(float(vals.median()), 3)
    stats['квартилі'] = [round(float(q), 3) for q in vals.quantile([0.25, 0.75])]
    stats['середнє'] = round(float(vals.mean()), 3)
    stats['ст. відхилення'] = round(float(vals.std()), 3)
    stats['коеф. варіації, %'] = round(100 * float(vals.std()) / float(vals.mean()), 1)
    if (vals > 0).all():
        stats['геом. середнє'] = round(float(np.exp(np.log(vals).mean())), 3)
    return stats


for c in MY_COLS:
    print(f'--- {c}  ({PASSPORT[c][0]})')
    for key, value in allowed_stats(c, PASSPORT[c][0]).items():
        print(f'      {key:<22}: {value}')
    print()
```

```
--- Телефон  (ідентифікатор)
      непорожніх            : 120
      унікальних            : 120
      частка унікальних     : 1.0
      найбільша частота     : 1
      можна                 : порівняння на рівність і пошук дублікатів
      про моду              : формально рахується, але беззмістовна: найчастіше значення трапляється 1 раз(и)
      не можна              : медіана, середнє, будь-яка арифметика над значеннями

--- Стать  (номінальна)
      непорожніх            : 121
      унікальних            : 2
      мода                  : ж
      частоти               : {'ж': 75, 'ч': 46}
      не можна              : медіана й середнє — порядку між категоріями немає

--- ступінь КТ  (порядкова)
      непорожніх            : 42
      унікальних            : 5
      мода                  : середній
      частоти               : {'середній': 18, 'важкий': 10, 'легкий': 7, 'вкрай важкий': 4, 'критичний': 3}
      медіана               : середній
      квартилі              : ['середній', 'важкий']
      не можна              : середнє й СКВ — відстані між сусідніми рівнями невідомі

--- Виписка  (інтервальна)
      непорожніх            : 118
      унікальних            : 88
      мода                  : 2021-09-07 00:00:00
      медіана               : 2022-03-15 00:00:00
      середнє               : 2022-01-29 18:06:06.101695
      ст. відхилення, днів  : 109.52
      розмах, днів          : 421
      не можна              : коефіцієнт варіації й відношення — нуль календаря умовний

--- КТ  (абсолютна)
      непорожніх            : 43
      унікальних            : 21
      мода                  : 30.0
      медіана               : 35.0
      квартилі              : [26.25, 63.75]
      середнє               : 42.5
      ст. відхилення        : 21.966
      коеф. варіації, %     : 51.7
      геом. середнє         : 36.071

--- Т вище 37,5  (відношень)
      непорожніх            : 78
      унікальних            : 9
      мода                  : 3.0
      частоти               : {3.0: 29, 2.0: 12, 5.0: 9, 4.0: 9, 7.0: 7, 6.0: 5, 10.0: 4, 1.0: 2, 14.0: 1}
      медіана               : 3.0
      квартилі              : [3.0, 5.0]
      середнє               : 4.192
      ст. відхилення        : 2.358
      коеф. варіації, %     : 56.2
      геом. середнє         : 3.677

--- Сечов2  (відношень)
      непорожніх            : 101
      унікальних            : 65
      мода                  : 4.9
      медіана               : 7.3
      квартилі              : [5.9, 10.0]
      середнє               : 10.199
      ст. відхилення        : 13.275
      коеф. варіації, %     : 130.2
      геом. середнє         : 8.024

--- БРпр1  (відношень)
      непорожніх            : 109
      унікальних            : 20
      мода                  : 3.4
      медіана               : 5.1
      квартилі              : [3.4, 5.3]
      середнє               : 4.801
      ст. відхилення        : 1.575
      коеф. варіації, %     : 32.8
      геом. середнє         : 4.566
```

Що в цьому виводі варте уваги

- `Телефон` не отримав навіть моди. Мода ідентифікатора — це «номер, який трапився найчастіше»; за означенням ідентифікатора він трапляється рівно один раз, тож число було б, а змісту в ньому — нуль.
- `ступінь КТ` отримав медіану **«середній»** і квартилі **«середній — важкий»**. Це назви рівнів, а не числа: медіана порядкової шкали — той рівень, нижче й вище якого лежить по половині спостережень, і повертати її треба саме рівнем.
- `Виписка` отримала середню дату (різниці дат дозволені), але **не коефіцієнт варіації**. Чому саме — показано в розділі 6 етапу 6.
- `Сечов2` має коефіцієнт варіації **130%** при медіані 7.3 — ознака того, що в стовпці сидять значення в чужій одиниці вимірювання. Розбираю це на етапі 3.

Етап 3. Три безглузді числа

`df.describe()` порахує статистики для всього, що схоже на число. Частина цих чисел не має жодного змісту.

Знайдіть **щонайменше три** такі числа у своєму наборі: обчисліть їх, а потім поясніть, чому кожне з них безглузде.

In [11]:
```
df[MY_COLS].describe(include='all').T
```

Out[11]:

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
Телефон	120	120	0958469788 дружина Ганна	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Стать	121	4	ж	56	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ступінь КТ	42	12	середній	10	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Виписка	120	90	2021-11-19 00:00:00	3	NaN	NaN	NaN	NaN	NaN	NaN	NaN
КТ	44.0	26.0	0.3	6.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Т вище 37,5	78.0	35.0	3.0	13.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Сечов2	102.0	66.0	5.0	4.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
БРпр1	109.0	NaN	NaN	NaN	4.801468	1.575191	2.4	3.4	5.1	5.3	10.2

In [12]:
```
# Рахую те, що pandas видасть без жодної скарги — саме так, як це зробив би недбалий аналіз

# 1. середній номер телефону
phone_num   = pd.to_numeric(df['Телефон'], errors='coerce')
mean_phone  = phone_num.mean()

# 2. «кількість статей» = насправді кількість написань
n_sexes     = df['Стать'].nunique()

# 3. середній ступінь КТ за алфавітними кодами (такий порядок дає value_counts().sort_index())
alpha_codes     = {lvl: i for i, lvl in enumerate(sorted(df['ступінь КТ'].dropna().unique()))}
mean_degree  = df['ступінь КТ'].map(alpha_codes).mean()

# 4. середній обсяг ураження легень «в лоб»
ct_naive    = pd.to_numeric(df['КТ'], errors='coerce')
mean_ct       = 100 * ct_naive.mean()

# 5. середня сечовина разом із викидами
urea        = pd.to_numeric(df['Сечов2'], errors='coerce')
mean_urea    = urea.mean()
cv_urea     = 100 * urea.std() / urea.mean()

print(f'1. середній номер телефону        : {mean_phone:,.1f}  (з {phone_num.notna().sum()} значень зі 120)')
print(f'2. «кількість статей» у вибірці   : {n_sexes}')
print(f'3. середній ступінь КТ (алф. коди): {mean_degree:.2f} -> {sorted(alpha_codes)
[int(round(mean_degree))]!r}')
print(f'4. середній обсяг ураження легень : {mean_ct:.1f} %  (з {ct_naive.notna().sum()} пацієнтів зі
{len(df)})')
print(f'5. середня сечовина               : {mean_urea:.2f}  при медіані {urea.median()} і CV {cv_urea:.0f}%')
```

```
1. середній номер телефону        : 758,798,950.4  (з 23 значень зі 120)
2. «кількість статей» у вибірці   : 4
3. середній ступінь КТ (алф. коди): 5.90 -> 'легкий'
4. середній обсяг ураження легень : 39.6 %  (з 27 пацієнтів зі 121)
5. середня сечовина               : 10.20  при медіані 7.3 і CV 130%
```

In [13]:
```python
# Доказ до №4: текстові клітинки КТ систематично важчі, тож to_numeric губить їх не випадково
is_number = df['КТ'].map(lambda v: isinstance(v, (int, float, np.integer, np.floating)) and pd.notna(v))
is_text = df['КТ'].map(lambda v: isinstance(v, str) and v.strip() != '')
ct_clean = CLEAN['КТ']

print(f'клітинки-числа : n={int(is_number.sum()):>3}   середнє {ct_clean[is_number].mean():.1f}%   медіана
{ct_clean[is_number].median():.1f}%')
print(f'клітинки-текст : n={int(is_text.sum()):>3}   середнє {ct_clean[is_text].mean():.1f}%   медіана
{ct_clean[is_text].median():.1f}%')
print(f'усе разом      : n={int(ct_clean.notna().sum()):>3}   середнє {ct_clean.mean():.1f}%   медіана
{ct_clean.median():.1f}%')
print()
print(f'Наївне середнє занижує обсяг ураження на {ct_clean.mean() - mean_ct:.1f} в.п.,')
print('бо викидає саме ті записи, де лікар писав діапазон — а писав він його для важких.')
print()
print('Доказ до №5: значення поза фізіологічним діапазоном сечовини (норма 2.5–8.3 ммоль/л):')
print('   ', sorted(urea[urea > 30].tolist()), '— це мг/дл або мкмоль/л, а не ммоль/л')
urea_no_outliers = urea[urea <= 30]
print(f'    без них: середнє {urea_no_outliers.mean():.2f}, CV
{100*urea_no_outliers.std()/urea_no_outliers.mean():.0f}% '
      f'(було {mean_urea:.2f} і {cv_urea:.0f}%)')
```

```
клітинки-числа : n= 27   середнє 39.6%   медіана 30.0%
клітинки-текст : n= 16   середнє 47.5%   медіана 45.0%
усе разом      : n= 43   середнє 42.5%   медіана 35.0%

Наївне середнє занижує обсяг ураження на 2.9 в.п.,
бо викидає саме ті записи, де лікар писав діапазон — а писав він його для важких.

Доказ до №5: значення поза фізіологічним діапазоном сечовини (норма 2.5–8.3 ммоль/л):
    [35.0, 37.0, 70.0, 116.0] — це мг/дл або мкмоль/л, а не ммоль/л
    без них: середнє 7.96, CV 41% (було 10.20 і 130%)
```

In [14]:
```python
MEANINGLESS = [
    # ('стовпець', 'яка статистика', значення, 'чому це число нічого не означає'),

    ('Телефон', 'середнє арифметичне', round(mean_phone, 1),
     'номер телефону — ідентифікатор, а не величина: цифри в ньому кодують оператора й абонента, '
     'а не кількість чогось, тож їхня сума не є величиною і ділити її ні на що; до того ж це середнє '
     'пораховано з 23 клітинок зі 120, бо решта 97 містять ще й ім’я родича й у число не перетворились'),

    ('Стать', 'nunique() як «кількість категорій»', n_sexes,
     'це кількість РІЗНИХ НАПИСАНЬ, а не категорій: «ч» і «чол» — та сама категорія, записана двома '
     'способами; реальних категорій дві (ж 75, ч 46), і будь-яка частотна таблиця по сирому стовпцю '
     'ділить кожну з них навпіл між її написаннями'),

    ('ступінь КТ', 'середнє числових кодів', round(mean_degree, 2),
     'подвійна безглуздість: коди роздано за алфавітом («важкий»=0, «середній»=11), тобто порядок '
     'кодів не має нічого спільного з порядком тяжкості; і навіть за правильного порядку середнє '
     'порядкової шкали не визначене, бо відстані між сусідніми рівнями невідомі й нерівні'),

    ('КТ', 'середнє після pd.to_numeric', round(mean_ct, 1),
     f'пораховано лише {int(ct_naive.notna().sum())} пацієнтів зі {len(df)}, бо 17 текстових '
     f'клітинок (16 діапазонів на кшталт «30-40%» і один рядок із самих пробілів) мовчки стали '
     f'NaN; викинуті записи систематично важчі '
     f'({ct_clean[is_text].mean():.1f}% проти {ct_clean[is_number].mean():.1f}%), тож число не просто '
     f'неповне — воно зміщене вниз'),

    ('Сечов2', 'середнє та коефіцієнт варіації', round(mean_urea, 2),
     f'CV={cv_urea:.0f}% при медіані {urea.median()} видає, що в стовпці змішано одиниці вимірювання: '
     'значення 35, 37, 70 і 116 фізіологічно неможливі для ммоль/л; середнє тягнеться за ними, '
     'і «10.2 ммоль/л» описує не пацієнтів, а помилку введення'),
]

assert len(MEANINGLESS) >= 3, 'Потрібно щонайменше три приклади'
pd.DataFrame(MEANINGLESS, columns=['Стовпець', 'Статистика', 'Значення', 'Чому безглузде'])
```

Out[14]:

	Стовпець	Статистика	Значення	Чому безглузде
0	Телефон	середнє арифметичне	758798950.4	номер телефону — ідентифікатор, а не величина: цифри в н...
1	Стать	nunique() як «кількість категорій»	4.0	це кількість РІЗНИХ НАПИСАНЬ, а не категорій: «ч» і «чол...
2	ступінь КТ	середнє числових кодів	5.9	подвійна безглуздість: коди роздано за алфавітом («важки...
3	КТ	середнє після pd.to_numeric	39.6	пораховано лише 27 пацієнтів зі 121, бо 17 текстових кліт...
4	Сечов2	середнє та коефіцієнт варіації	10.2	CV=130% при медіані 7.3 видає, що в стовпці змішано один...

Етап 4. Міра зв'язку для вашої пари

Для двох змінних вашого варіанта треба оцінити зв'язок. Міру обирає **пара шкал**, а не зручність:

Поєднання шкал	Міра
відношень × відношень	кореляція Пірсона
порядкова × порядкова	рангова кореляція Спірмена
номінальна × номінальна	χ^2 та коефіцієнт Крамера (V)
відношень × бінарна номінальна	точковий бісеріальний коефіцієнт
номінальна × порядкова	коефіцієнт Крамера (V)

Обґрунтуйте вибір, обчисліть міру й поясніть результат. Окремо покажіть, **що станеться**, якщо застосувати Пірсона там, де він недопустимий, — і чому отримане число не можна інтерпретувати.

In [15]:
```
A, B = VARIANT['пара']
print('Ваша пара:', A, '×', B)
print()
print(f'{A}: шкала —', PASSPORT.get(A, ('не у вашому списку',))[0] or '(визначте самостійно)')
print(f'{B}: шкала —', PASSPORT.get(B, ('не у вашому списку',))[0] or '(визначте самостійно)')
print()
print(df[[A, B]].head(6).to_string())
```

```
Ваша пара: Стать × ступінь КТ

Стать: шкала — номінальна
ступінь КТ: шкала — порядкова

  Стать ступінь КТ
0     ч    сердній
1     ч   середній
2     ж  критичний
3     ж  критичний
4     ж        NaN
5     ж   середній
```

In [16]:
```
CHOSEN_MEASURE = 'χ² та коефіцієнт Крамера (V)'         # напишіть назву міри
WHY_THIS_ONE = (                                    # одне-два речення
    'Стать — номінальна, ступінь КТ — порядкова; у таблиці методички поєднанню '
    '«номінальна × порядкова» відповідає саме коефіцієнт Крамера. Він рахується з частот '
    'у таблиці спряженості, а не зі значень, тому не вимагає від категорій ані числових кодів, '
    'ані рівних відстаней між рівнями — тобто не приписує даним того, чого в них немає.'
)

assert CHOSEN_MEASURE and len(WHY_THIS_ONE) >= 40, 'Заповніть обидва поля'

# ваш код: обчисліть обрану міру
sex_c = CLEAN['Стать']
degree_c = CLEAN['ступінь КТ']

table = pd.crosstab(sex_c, degree_c).reindex(columns=CT_ORDER, fill_value=0)
print('Таблиця спряженості (рівні — у порядку тяжкості, не за алфавітом):')
print(table.to_string())
print()


def cramer_v(tbl):
    # χ² Пірсона та V Крамера з таблиці спряженості.
    # χ² = Σ (спостережене − очікуване)² / очікуване, де очікуване — добуток маргінальних
    # частот, поділений на n (тобто частоти, які були б за незалежності).
    # V = sqrt(χ² / (n · (min(рядки, стовпці) − 1))) — нормування на [0, 1], завдяки якому
    # V не залежить ні від обсягу вибірки, ні від розміру таблиці.
    observed = tbl.values.astype(float)
```

```
    n = observed.sum()
    exp_freq = np.outer(observed.sum(axis=1), observed.sum(axis=0)) / n
    chi2 = ((observed - exp_freq) ** 2 / exp_freq).sum()
    V = np.sqrt(chi2 / (n * (min(observed.shape) - 1)))
    return chi2, V, n, exp_freq


chi2, V, n, expected = cramer_v(table)
dof = (table.shape[0] - 1) * (table.shape[1] - 1)

print(f'χ²  = {chi2:.4f}   (ступенів свободи {dof})')
print(f'V   = {V:.4f}   при n = {int(n)}')
print()
print(f'Клітинок з очікуваною частотою < 5: {int((expected < 5).sum())} з {expected.size} '
      f'(мінімальна очікувана {expected.min():.1f})')
```

```
Таблиця спряженості (рівні — у порядку тяжкості, не за алфавітом):
ступінь КТ  легкий  середній  важкий  вкрай важкий  критичний
Стать
ж                4         8       4             2          3
ч                3        10       6             2          0

χ²  = 3.7651   (ступенів свободи 4)
V   = 0.2994   при n = 42

Клітинок з очікуваною частотою < 5: 6 з 10 (мінімальна очікувана 1.5)
```

Як це читати

V = 0.299 — слабкий-помірний зв'язок. Напрямку в нього немає й бути не може: `V` завжди невід'ємний, бо «стать більша/менша» — порожня фраза. Він каже лише, **наскільки** розподіл тяжкості відрізняється між чоловіками й жінками.

P-значення я не наводжу, і це свідомо. Шість клітинок із десяти мають очікувану частоту менше 5 (мінімальна — 1.5), а наближення статистики χ^2 до розподілу хі-квадрат на таких частотах не працює. Число вийшло б, але довіряти йому не можна — це та сама помилка, з якої складається етап 6, тільки в іншому місці.

І головне обмеження тут не в мірі, а у вибірці. Наступна комірка показує, чому.

In [17]:
```
# Звідки взялися 42 пацієнти зі 121 — і чи можна переносити V на все відділення
has_degree = degree_c.notna()
print(f'ступінь КТ заповнено у {int(has_degree.sum())} пацієнтів зі {len(df)}')
print()

coverage = pd.crosstab(sex_c, has_degree, normalize='index').mul(100).round(1)
coverage.columns = ['ступеня немає, %', 'ступінь є, %']
print('Хто потрапив у ці 42 — у розрізі статі:')
print(coverage.to_string())
print()

mortality = (pd.crosstab(has_degree, df['помер/жив'].map({1: 'помер', 2: 'вижив'}), normalize='index')
             .mul(100).round(1))
mortality.index = ['ступеня немає', 'ступінь є']
print('Хто потрапив у ці 42 — у розрізі результату лікування:')
print(mortality.to_string())
print()
print(f'Підвибірка не випадкова: серед неї вижило {mortality.loc["ступінь є", "вижив"]:.0f}% '
      f'проти {mortality.loc["ступеня немає", "вижив"]:.0f}% серед решти,')
print(f'а опис ступеня мають {coverage.loc["ч", "ступінь є, %"]:.0f}% чоловіків проти '
      f'{coverage.loc["ж", "ступінь є, %"]:.0f}% жінок.')
print('V = 0.299 описує саме тих, хто дожив до опису КТ, — а не все відділення.')
```

```
ступінь КТ заповнено у 42 пацієнтів зі 121

Хто потрапив у ці 42 — у розрізі статі:
       ступеня немає, %  ступінь є, %
Стать
ж                  72.0          28.0
ч                  54.3          45.7

Хто потрапив у ці 42 — у розрізі результату лікування:
помер/жив      вижив  помер
ступеня немає   32.9   67.1
ступінь є       81.0   19.0

Підвибірка не випадкова: серед неї вижило 81% проти 33% серед решти,
а опис ступеня мають 46% чоловіків проти 28% жінок.
V = 0.299 описує саме тих, хто дожив до опису КТ, — а не все відділення.
```

In [18]:

```python
# Тепер — контрприклад: застосуйте кореляцію Пірсона незалежно від шкал
# і поясніть, чому отримане число не можна використовувати.

sex_codings = {'ч=0, ж=1': {'ч': 0, 'ж': 1},
               'ч=1, ж=0': {'ч': 1, 'ж': 0}}
degree_codings = {
    'за тяжкістю':  {level: i for i, level in enumerate(CT_ORDER)},
    'у зворотному': {level: i for i, level in enumerate(reversed(CT_ORDER))},
    'за алфавітом': {level: i for i, level in enumerate(sorted(CT_ORDER))},
}

print('Кореляція Пірсона на ТИХ САМИХ 42 пацієнтах, різні кодування категорій:')
print()
print(f'{"кодування статі":<18}{"кодування ступеня":<18}{"r Пірсона":>12}{"V Крамера":>12}')
print('-' * 60)
for sex_label, sex_code in sex_codings.items():
    for degree_label, degree_code in degree_codings.items():
        r = sex_c.map(sex_code).corr(degree_c.map(degree_code))
        # V рахую з тієї самої перекодованої таблиці — щоб порівняння було чесним
        _, V_recoded, _, _ = cramer_v(pd.crosstab(sex_c.map(sex_code), degree_c.map(degree_code)))
        print(f'{sex_label:<18}{degree_label:<18}{r:>+12.4f}{V_recoded:>12.4f}')
print()
print('Шість чисел від -0.1303 до +0.1303 — і жодне з них не є властивістю даних.')
print('V у правому стовпці не ворухнувся: 0.2994 у всіх шести рядках.')
```

```
Кореляція Пірсона на ТИХ САМИХ 42 пацієнтах, різні кодування категорій:

кодування статі   кодування ступеня    r Пірсона   V Крамера
------------------------------------------------------------
ч=0, ж=1          за тяжкістю            +0.1303      0.2994
ч=0, ж=1          у зворотному           -0.1303      0.2994
ч=0, ж=1          за алфавітом           +0.0144      0.2994
ч=1, ж=0          за тяжкістю            -0.1303      0.2994
ч=1, ж=0          у зворотному           +0.1303      0.2994
ч=1, ж=0          за алфавітом           -0.0144      0.2994

Шість чисел від -0.1303 до +0.1303 — і жодне з них не є властивістю даних.
V у правому стовпці не ворухнувся: 0.2994 у всіх шести рядках.
```

Чому ці шість чисел нічого не варті

Дані в усіх шести рядках **однакові** — ті самі 42 пацієнти, та сама таблиця спряженості. Змінювались лише цифри, якими я позначив категорії. І результат поїхав:

- **знак** перевертається, щойно поміняти місцями `ч` і `ж`. Пірсон стверджує напрямок зв'язку, а напрямок тут — мій довільний вибір, хто з них «нуль»;
- **величина** падає вдев'ятеро (0.1303 → 0.0144), якщо впорядкувати рівні тяжкості за алфавітом замість порядку тяжкості. Саме так, нагадаю, і вчинив помилковий звіт у розділі 3.

Причина в тому, що `r` будується з середніх і стандартних відхилень **самих значень**, а для номінальної шкали значення — це мітки. Міняєш мітки — міняється `r`; осмислена ж міра зв'язку мусить залежати від даних, а не від того, як їх занумерували.

`V` Крамера будується з **частот у клітинках**, і в правому стовпці таблиці видно наслідок: перестановка кодів не зачіпає його взагалі. Тому 0.2994 — це властивість даних, а шість

значень `r` — властивість мого `dict`.

Етап 5. Підготовка змінних до моделі

Шкала визначає не лише статистику, а й спосіб кодування.

Шкала	Як готують до моделі
Номінальна	one-hot кодування; порядок категорій не вводять
Порядкова	впорядкована категорія або цілі коди із **заданим** порядком
Інтервальна	стандартизація; для дат — перехід до тривалостей
Відношень	нормалізація або стандартизація; можливе логарифмування
Ідентифікатор	у модель не беруть узагалі

Для кожного свого стовпця вкажіть спосіб підготовки — і **виконайте його** щонайменше для трьох стовпців різних шкал.

In [19]:
```
PREPARATION = {
    # 'стовпець': 'спосіб і коротке обґрунтування',

    'Телефон':
        'У модель не беремо взагалі. Ідентифікатор: 120 унікальних значень на 120 заповнених '
        'клітинок — будь-яка модель використає його як номер рядка й ідеально запам’ятає навчальну '
        'вибірку, не навчившись нічого. Одна похідна ознака все ж допустима — прапорець «вказано '
        'контакт родича» (у 97 зі 120 клітинок є ім’я), бо це вже властивість пацієнта, а не мітка.',

    'Стать':
        'Звести чотири написання до двох категорій (clean_sex), далі one-hot. Категорій рівно дві, '
        'тож достатньо одного бінарного стовпця «стать_ж»: другий був би його точним доповненням '
        'і створив би колінеарність. Порядку не вводимо — його немає.',

    'ступінь КТ':
        'Впорядкована категорія pd.Categorical(ordered=True) з CT_ORDER, далі цілі коди 0..4. '
        'Порядок задаємо руками, бо алфавіт його не знає. Коди годяться лише для моделей, що читають '
        'їх як порядок (дерева, монотонні моделі), і не для лінійної регресії, яка прийняла б різницю '
        'між легким і середнім за ту саму, що й між вкрай важким і критичним — а в даних ці кроки '
        'дорівнюють 15.6 і 4.5 в.п. Пропуск (79 зі 121) кодуємо окремим прапорцем, а не нулем.',

    'Виписка':
        'Сама дата в модель не йде: вона інтервальна й кодує календар, а не пацієнта — модель '
        'вивчила б хвилі захворюваності. Переходимо до тривалості: (Виписка − Поступлення) у днях. '
        'Тривалість уже шкала відношень, її стандартизуємо. Два рядки з «24.024.2021» і дві '
        'від’ємні тривалості позначаємо як пропуск, а не правимо навмання.',

    'КТ':
        'Звести обидва кодування до відсотків (clean_ct), діапазон — серединою. Шкала абсолютна й '
        'уже обмежена [0, 100], тож min-max нічого не додає: ділимо на 100 і отримуємо частку. '
        'Пропуски (77 зі 121) не випадкові — супроводжуємо окремим прапорцем «КТ не описано».',

    'Т вище 37,5':
        'Розпадається на ДВІ ознаки: число днів (fever_days, шкала відношень, стандартизуємо) і '
        'період (fever_period, номінальна, one-hot). Без другої перша непорівнювана: «3 дні в стац» '
        'і «3 дні до госп» дають однакове число за різних клінічних ситуацій.',

    'Сечов2':
        'pd.to_numeric (рядок «немає» → NaN), далі позначити значення > 30 ммоль/л як чужу одиницю '
        'вимірювання й перевести в пропуск. Розподіл правобічно скошений, шкала відношень — '
        'логарифмуємо (log1p) і стандартизуємо; логарифм тут допустимий саме тому, що нуль природний.',

    'БРпр1':
        'Шкала відношень, стовпець чистий (109 зі 121, одне кодування). Стандартизація z = (x − μ)/σ; '
        'за потреби логарифмування, бо розподіл скошений (середнє 4.80 при медіані 5.10 і максимумі '
        '10.2). Min-max не беремо: він прив’язав би шкалу до єдиного максимального значення.',
}
for c in MY_COLS:
    PREPARATION.setdefault(c, '')
```

```
for c in MY_COLS:
    print(f'--- {c}  ({PASSPORT[c][0]})')
    print(textwrap.fill(PREPARATION[c], width=92,
                        initial_indent='      ', subsequent_indent='      '))
    print()
```

```
--- Телефон  (ідентифікатор)
      У модель не беремо взагалі. Ідентифікатор: 120 унікальних значень на 120 заповнених
      клітинок — будь-яка модель використає його як номер рядка й ідеально запам'ятає
      навчальну вибірку, не навчившись нічого. Одна похідна ознака все ж допустима —
      прапорець «вказано контакт родича» (у 97 зі 120 клітинок є ім'я), бо це вже
      властивість пацієнта, а не мітка.

--- Стать  (номінальна)
      Звести чотири написання до двох категорій (clean_sex), далі one-hot. Категорій рівно
      дві, тож достатньо одного бінарного стовпця «стать_ж»: другий був би його точним
      доповненням і створив би колінеарність. Порядку не вводимо — його немає.

--- ступінь КТ  (порядкова)
      Впорядкована категорія pd.Categorical(ordered=True) з CT_ORDER, далі цілі коди 0..4.
      Порядок задаємо руками, бо алфавіт його не знає. Коди годяться лише для моделей, що
      читають їх як порядок (дерева, монотонні моделі), і не для лінійної регресії, яка
      прийняла б різницю між легким і середнім за ту саму, що й між вкрай важким і критичним
      — а в даних ці кроки дорівнюють 15.6 і 4.5 в.п. Пропуск (79 зі 121) кодуємо окремим
      прапорцем, а не нулем.

--- Виписка  (інтервальна)
      Сама дата в модель не йде: вона інтервальна й кодує календар, а не пацієнта — модель
      вивчила б хвилі захворюваності. Переходимо до тривалості: (Виписка − Поступлення) у
      днях. Тривалість уже шкала відношень, її стандартизуємо. Два рядки з «24.024.2021» і
      дві від'ємні тривалості позначаємо як пропуск, а не правимо навмання.

--- КТ  (абсолютна)
      Звести обидва кодування до відсотків (clean_ct), діапазон — серединою. Шкала абсолютна
      й уже обмежена [0, 100], тож min-max нічого не додає: ділимо на 100 і отримуємо
      частку. Пропуски (77 зі 121) не випадкові — супроводжуємо окремим прапорцем «КТ не
      описано».

--- Т вище 37,5  (відношень)
      Розпадається на ДВІ ознаки: число днів (fever_days, шкала відношень, стандартизуємо) і
      період (fever_period, номінальна, one-hot). Без другої перша непорівнювана: «3 дні в
      стац» і «3 дні до госп» дають однакове число за різних клінічних ситуацій.

--- Сечов2  (відношень)
      pd.to_numeric (рядок «немає» → NaN), далі позначити значення > 30 ммоль/л як чужу
      одиницю вимірювання й перевести в пропуск. Розподіл правобічно скошений, шкала
      відношень — логарифмуємо (log1p) і стандартизуємо; логарифм тут допустимий саме тому,
      що нуль природний.

--- БРпр1  (відношень)
      Шкала відношень, стовпець чистий (109 зі 121, одне кодування). Стандартизація z = (x −
      μ)/σ; за потреби логарифмування, бо розподіл скошений (середнє 4.80 при медіані 5.10 і
      максимумі 10.2). Min-max не беремо: він прив'язав би шкалу до єдиного максимального
      значення.
```

In [20]:

```
# ваш код: реалізуйте підготовку для трьох стовпців різних шкал
X = pd.DataFrame(index=df.index)

# ── 1. НОМІНАЛЬНА: Стать -> one-hot (для двох категорій досить одного стовпця)
X['стать_ж'] = (CLEAN['Стать'] == 'ж').astype('Int64')

# ── 2. ПОРЯДКОВА: ступінь КТ -> впорядкована категорія -> коди 0..4, пропуск окремим прапорцем
degree_cat = pd.Categorical(CLEAN['ступінь КТ'], categories=CT_ORDER, ordered=True)
degree_codes = pd.Series(degree_cat.codes, index=df.index)
X['ступінь_код'] = degree_codes.where(degree_codes >= 0).astype('Int64')
X['ступінь_невідомий'] = CLEAN['ступінь КТ'].isna().astype('Int64')

# ── 3. ІНТЕРВАЛЬНА: Виписка -> тривалість у днях -> стандартизація
duration = (CLEAN['Виписка'] - pd.to_datetime(df['Поступлення'], errors='coerce')).dt.days
duration = duration.where(duration.between(0, 120))   # від'ємні й 194 дні — явні помилки дат
X['ліжко_днів_z'] = (duration - duration.mean()) / duration.std()

# ── 4. АБСОЛЮТНА: КТ -> відсотки -> частка [0, 1] (min-max зайвий, межі шкали відомі заздалегідь)
X['кт_частка'] = CLEAN['КТ'] / 100
X['кт_невідомо'] = CLEAN['КТ'].isna().astype('Int64')

# ── 5. ВІДНОШЕНЬ: Сечов2 -> прибрати чужу одиницю -> log1p -> стандартизація
urea_clean = CLEAN['Сечов2'].where(CLEAN['Сечов2'] <= 30)
log_urea = np.log1p(urea_clean)
X['сечовина_log_z'] = (log_urea - log_urea.mean()) / log_urea.std()

# ── 6. ВІДНОШЕНЬ: БРпр1 -> стандартизація
```

```
X['брпр1_z'] = (CLEAN['БРпр1'] - CLEAN['БРпр1'].mean()) / CLEAN['БРпр1'].std()

# ── 7. Т вище 37,5 розпадається на число днів + період
X['днів_гарячки'] = CLEAN['Т вище 37,5']
X = X.join(pd.get_dummies(FEVER_PERIOD.fillna('запису немає'),
                          prefix='гарячка').astype('Int64'))

# ── Телефон у X не потрапив узагалі — і це рішення, а не недогляд
print('Матриця ознак:', X.shape, '| Телефон у ній відсутній свідомо')
print()
print(X.describe().T.round(3).to_string())
```

```
Матриця ознак: (121, 14) | Телефон у ній відсутній свідомо

                          count      mean       std       min       25%       50%       75%       max
стать_ж                   121.0  0.619835  0.487446       0.0       0.0       1.0       1.0       1.0
ступінь_код                42.0   1.47619  1.109561       0.0       1.0       1.0       2.0       4.0
ступінь_невідомий         121.0  0.652893   0.47803       0.0       0.0       1.0       1.0       1.0
ліжко_днів_z              115.0       0.0       1.0 -1.231472 -0.620715 -0.162647  0.295421  4.570721
кт_частка                  43.0     0.425  0.219662      0.05    0.2625      0.35    0.6375       0.8
кт_невідомо               121.0  0.644628  0.480616       0.0       0.0       1.0       1.0       1.0
сечовина_log_z             97.0      -0.0       1.0 -1.970736 -0.614772  -0.09455  0.621748   2.93411
брпр1_z                   109.0      -0.0       1.0 -1.524557 -0.889713  0.189521   0.31649  3.427225
днів_гарячки               78.0  4.192308  2.357953       1.0       3.0       3.0       5.0      14.0
гарячка_до госпіталізації 121.0  0.198347  0.400413       0.0       0.0       0.0       0.0       1.0
гарячка_запису немає      121.0  0.355372  0.480616       0.0       0.0       0.0       1.0       1.0
гарячка_змішаний          121.0  0.008264  0.090909       0.0       0.0       0.0       0.0       1.0
гарячка_не вказано        121.0  0.247934  0.433609       0.0       0.0       0.0       0.0       1.0
гарячка_у стаціонарі      121.0  0.190083  0.393998       0.0       0.0       0.0       0.0       1.0
```

In [21]:
```
# Перевірка, що підготовка не зіпсувала шкали
print('Стать:       ', X['стать_ж'].value_counts().to_dict(), '— дві категорії, не чотири')
print()
print('ступінь_код: кожному рівню відповідає рівно один код, і коди зростають з тяжкістю:')
mapping = (X.loc[CLEAN['ступінь КТ'].notna(), 'ступінь_код']
             .groupby(CLEAN['ступінь КТ']).unique().reindex(CT_ORDER))
for level, code in mapping.items():
    print(f'    {level:<14} -> {list(code)}')
print()
print('Стандартизовані стовпці (мають дати середнє ≈ 0 і СКВ ≈ 1):')
for c in ['ліжко_днів_z', 'сечовина_log_z', 'брпр1_z']:
    print(f'    {c:<16} середнє {X[c].mean():+.3f}   СКВ {X[c].std():.3f}   непорожніх
{int(X[c].notna().sum())}')
print()
print(f'кт_частка лежить у [0, 1] без жодної нормалізації: '
      f'[{X["кт_частка"].min():.2f}, {X["кт_частка"].max():.2f}]')
```

```
Стать:        {np.int64(1): 75, np.int64(0): 46} — дві категорії, не чотири

ступінь_код: кожному рівню відповідає рівно один код, і коди зростають з тяжкістю:
    легкий         -> [np.int64(0)]
    середній       -> [np.int64(1)]
    важкий         -> [np.int64(2)]
    вкрай важкий   -> [np.int64(3)]
    критичний      -> [np.int64(4)]

Стандартизовані стовпці (мають дати середнє ≈ 0 і СКВ ≈ 1):
    ліжко_днів_z     середнє +0.000   СКВ 1.000   непорожніх 115
    сечовина_log_z   середнє -0.000   СКВ 1.000   непорожніх 97
    брпр1_z          середнє -0.000   СКВ 1.000   непорожніх 109

кт_частка лежить у [0, 1] без жодної нормалізації: [0.05, 0.80]
```

Перед нормалізацією обов'язково перевірте варіативність. Стовпець, у якому всі значення однакові, дасть нуль у знаменнику — і мовчазний `NaN` замість помилки.

In [22]:
```
for c in MY_COLS:
    vals = pd.to_numeric(df[c], errors='coerce').dropna()
    if len(vals) and vals.nunique() == 1:
        print(f'УВАГА: {c} — сталий стовпець, нормалізація дасть ділення на нуль')
    elif len(vals) and vals.max() == vals.min():
        print(f'УВАГА: {c} — max дорівнює min')
print('Перевірку варіативності виконано.')
```

```
Перевірку варіативності виконано.
```

In [23]:
```
# У моїх восьми стовпцях сталих немає — але у файлі вони є.
# Та сама перевірка по всьому набору знаходить рівно те, на чому мовчки зламався розділ 8 звіту:
numeric = df.select_dtypes('number')
for c in numeric.columns:
    vals = numeric[c].dropna()
    if len(vals) and vals.max() == vals.min():
        print(f'УВАГА: {c} — n={len(vals)}, усі значення = {vals.iloc[0]}, '
              f'max == min  ->  (x-min)/(max-min) = 0/0 = NaN')
```

```
print()
print('Жоден assert при цьому не спрацює: pandas поверне NaN і піде далі. Перевіряю:')
print()

constant = df['ФЕРИТ2']
naive = (constant - constant.min()) / (constant.max() - constant.min())
print(f'   (x - min)/(max - min) на ФЕРИТ2 -> непорожніх {int(naive.notna().sum())}, '
      f'усе інше NaN, винятку немає')


def safe_normalize(s):
    # Повертає NaN-стовпець лише свідомо: якщо розмах нульовий, нормалізувати нічого.
    value_range = s.max() - s.min()
    if pd.isna(value_range) or value_range == 0:
        raise ValueError(f'сталий стовпець (усі значення = {s.dropna().iloc[0]}), '
                         f'нормалізація не визначена')
    return (s - s.min()) / value_range


try:
    safe_normalize(constant)
except ValueError as e:
    print(f'   safe_normalize(ФЕРИТ2) -> ValueError: {e}')
print(f'   safe_normalize(Вік)      -> ok, діапазон '
      f'[{safe_normalize(df["Вік"]).min():.2f}, {safe_normalize(df["Вік"]).max():.2f}]')
```

```
УВАГА: ФЕРИТ2 — n=1, усі значення = 1265.0, max == min  ->  (x-min)/(max-min) = 0/0 = NaN

Жоден assert при цьому не спрацює: pandas поверне NaN і піде далі. Перевіряю:

   (x - min)/(max - min) на ФЕРИТ2 -> непорожніх 0, усе інше NaN, винятку немає
   safe_normalize(ФЕРИТ2) -> ValueError: сталий стовпець (усі значення = 1265.0), нормалізація не визначена
   safe_normalize(Вік)      -> ok, діапазон [0.00, 1.00]
```

In [24]:
```
# --- САМОПЕРЕВІРКА 2 ---
assert all(len(PREPARATION[c].strip()) >= 15 for c in MY_COLS), \
    'Для кожного стовпця вкажіть спосіб підготовки'
assert len(MEANINGLESS) >= 3, 'Етап 3 не завершено'
print('Самоперевірка 2 пройдена.')
```

```
Самоперевірка 2 пройдена.
```

Етап 6. Виправлення чужого аналізу

Відкрийте файл **`ЛР3_помилковий_аналіз.ipynb`**. Це звіт, який виконується без жодної помилки й містить десять хибних місць.

Для кожного з десяти заповніть рядок нижче:

- **розділ** — номер розділу звіту;
- **що не так** — яка шкала порушена або яке припущення хибне;
- **чому число неправильне** — що саме воно означає насправді;
- **як правильно** — коректний варіант або обґрунтована відмова від показника.

In [25]:
```
CORRECTIONS = [
    # (розділ, 'що не так', 'чому число неправильне', 'як правильно'),

    (1, 'Середнє від номінальної шкали: «помер/жив» містить мітки 1 і 2, а не величину',
     'Цифри 1 і 2 — назви двох результатів лікування, а не їхня кількість. Жоден пацієнт не був '
     '«посередині між летальним і сприятливим результатом»: 61 помер, 60 вижило. 1.50 переказує лише '
     'те, що групи майже рівні; заміна кодів на 0/1 дала б 0.50, на 10/20 — 15.0, і дані при цьому '
     'не змінилися б жодного разу',
     'Для номінальної шкали — тільки частоти й мода: померло 61 (50.4%), вижило 60 (49.6%). Якщо '
     'потрібне одне число, це летальність, тобто частка однієї категорії, а не середнє кодів'),

    (2, 'Частоти пораховано по сирому стовпцю: nunique() лічить написання, а не категорії',
     '«Чотирьох статей» не існує — «ч» і «чол» це та сама категорія, записана двома способами, так '
     'само «ж» і «жін». Кожна з двох реальних категорій розірвана навпіл між своїми написаннями, '
     'тому й «найбільша група ж = 56 осіб (46%)» хибна: жінок насправді 75 (62%), чоловіків 46 (38%)',
     'Спершу звести написання до категорій за першою літерою, і лише потім рахувати частоти. '
     'Будь-яка статистика категорій, порахована до зведення, описує охайність вводу, а не пацієнтів'),

    (3, 'Порядкова шкала: алфавітний порядок видано за порядок тяжкості, та ще й узято середнє',
```

```
     'value_counts().sort_index() сортує рядки за абеткою, тому «найлегшим» став «важкий» (перший '
     'за алфавітом), а «найважчим» — «середній» (останній). На тих самих алфавітних кодах узято '
     'середнє 5.9, і воно вказало на «легкий». Жодне з трьох тверджень не стосується тяжкості: '
     '12 рівнів тут — це 5 справжніх, розмножених друкарськими помилками й кінцевими пробілами',
     'Звести 12 написань до 5 рівнів, задати порядок руками через pd.Categorical(ordered=True) '
     'і брати медіану, а не середнє: медіана = «середній», квартилі «середній»–«важкий»'),

    (4, 'Число обчислене на непридатних даних: у стовпці КТ два несумісні кодування',
     'pd.to_numeric мовчки перетворив на NaN усі 17 текстових клітинок — 16 діапазонів («30-40%», '
     '«до 17%») і один рядок із самих пробілів, — тож «39.6%» описує 27 пацієнтів зі 121, '
     'тобто 22% відділення. Втрата не випадкова: саме в текстових '
     'клітинках лежать важчі випадки (середнє 47.5% проти 39.6% у числових), тому середнє ще й зміщене '
     'вниз. Розрив між середнім 39.6% і медіаною 30.0% має ту саму причину',
     'Розібрати обидва кодування в один масштаб (частка × 100, діапазон — серединою). Тоді описано '
     '43 пацієнти, середнє 42.5%, медіана 35.0%. Поруч обов’язково вказувати, що КТ описано лише '
     'у 44 зі 121 — число стосується цієї підвибірки, а не відділення'),

    (5, 'Число обчислене на непридатних даних: у spO2 змішано відсотки й частки',
     'Мінімум 0.9 — це не сатурація 0.9%, несумісна з життям, а частка 0.9, тобто 90%, записана '
     'в іншому кодуванні. Одне таке значення тягне середнє вниз, і «82.05%» не описує ні справжню '
     'середню сатурацію, ні «виражену дихальну недостатність у більшості»: медіана 84% і тут чесніша '
     'за середнє, бо на поодиноке чуже значення не реагує',
     'Знайти значення < 2 і домножити на 100 (або позначити пропуском), і лише потім рахувати. '
     'Межі фізіологічного діапазону (70–100%) мають перевірятися до будь-якої статистики'),

    (6, 'Коефіцієнт варіації від інтервальної шкали: у дат немає природного нуля',
     'CV = σ/μ має зміст лише тоді, коли нуль шкали природний, інакше знаменник залежить від того, '
     'звідки ми почали рахувати. Тут μ — це середня відстань від 1 січня 1970 року, близько 19 000 '
     'днів, і вона не є властивістю госпіталізацій. Доказ: від епохи 1970 CV = 0.595%, від 1900 року '
     '— 0.25%, від 1 січня 2021 — 30.0%. Одні й ті самі дати, CV відрізняється у 120 разів, тож '
     '«надзвичайна однорідність» — властивість обраної точки відліку, а не потоку пацієнтів',
     'Для інтервальної шкали CV не рахувати взагалі. Розсіяння дат описувати в одиницях різниці: '
     'розмах 463 дні (25.02.2021 – 03.06.2022), стандартне відхилення близько 113 днів — а це вже '
     'не однорідність, а півтора року роботи відділення'),

    (7, 'Пірсон між двома номінальними змінними, і висновок зроблено протилежний числу',
     'r = 0.177 не можна читати як «лінійний зв’язок»: між двома наборами міток прямої не існує, '
     'а знак визначається тим, кого позначили нулем — заміна ч=0,ж=1 на ч=1,ж=0 дає −0.177 на тих '
     'самих даних. Але й за величиною висновок хибний: для таблиці 2×2 |r| збігається з V Крамера, '
     'і 0.177 — це не «відсутність зв’язку», а помітна різниця: летальність 57.3% у жінок проти '
     '39.1% у чоловіків, тобто 18 в.п., $\chi^2$ = 3.78 при одному ступені свободи ($p \approx 0.052$)',
     'Таблиця спряженості + $\chi^2$ і V Крамера (V = 0.177), а для 2×2 ще інформативніше — різниця ризиків '
     '(18.2 в.п.) і відношення шансів (OR = 2.09). Ці міри не залежать від того, якими цифрами '
     'позначено категорії'),

    (8, 'Нормалізація застосована до всього, що має числовий тип, без перевірки варіативності й шкали',
     'select_dtypes("number") дав 40 стовпців, серед яких ФЕРИТ2 має єдине непорожнє значення 1265: '
     'max == min, формула (x − min)/(max − min) дає 0/0 = NaN, і pandas мовчки лишає порожній стовпець '
     'замість помилки — у виводі describe() це видно як count = 0.0 і NaN у кожному рядку. Туди ж '
     'потрапив «помер/жив»: нормалізувати мітки 1 і 2 до 0 і 1 технічно можна, але отриманий стовпець '
     'не стає величиною. Теза «усі значення приведено до діапазону [0,1]» хибна буквально',
     'Перед нормалізацією відсіяти стовпці з max == min і стовпці, числові лише за типом (коди, '
     'ідентифікатори). Нормалізувати поіменний список ознак шкали відношень, а не все, що int або float'),

    (9, 'Безумовний dropna() на наборі, де пропуск у частині стовпців структурний',
     'Лишилось 0 рядків зі 121, і «повністю заповнений набір» — це порожня таблиця. Причина не в '
     'поганих даних: у ФЕРИТ2 заповнено 1 значення, у ФЕРИТ1 — 2, в ІЛ6-2 — 6, а «Дата розтину» '
     'порожня у всіх 60 вижившим за визначенням — там немає чого заповнювати. Жоден рядок не має '
     'усіх 77 значень, тому перетин завжди порожній',
     'dropna() застосовувати лише до поіменного списку ознак, які справді підуть у модель, через '
     'subset=. Стовпці з часткою пропусків понад 50% виключати цілком, структурні пропуски кодувати '
     'окремою категорією або прапорцем, решту заповнювати медіаною'),

    (10, 'Витік цільової змінної: «Дата розтину» — наслідок смерті, а не її передвісник',
     'r = 1.00 не означає сильного предиктора — він означає, що стовпець є самою ціллю в іншому '
     'записі: дата розтину заповнена рівно у 61 померлого і порожня рівно у 60 вижившим. До того ж '
     'кореляція рахується не з дат: .astype("int64") перетворив NaT на −9.22·10^18, тож фактично '
     'корелюють «тут стоїть службове число» з «пацієнт вижив». Модель на таких ознаках дасть 100% '
     'на навчальній вибірці й буде непридатна для прогнозу — на момент прогнозу розтину ще немає. '
     'Решту рейтингу теж не можна зіставляти: кожне r пораховане на своїй підмножині пацієнтів',
     'Викинути з кандидатів усе, що фізично виникає після результату (Дата розтину, Виписка, ЛН, '
     'Ліжко днів). Для бінарної цілі брати точковий бісеріальний коефіцієнт або AUC, рахувати їх '
     'на спільній підмножині й поруч указувати n, на якому пораховано кожен показник'),
]

assert len(CORRECTIONS) == 10, f'Знайдено {len(CORRECTIONS)} помилок із 10'
for row in CORRECTIONS:
    assert len(row) == 4 and all(str(x).strip() for x in row), 'Заповніть усі чотири поля'
    assert len(str(row[2])) >= 30 and len(str(row[3])) >= 25, 'Пояснення закороткі'

def _block(label, text):
    print(textwrap.fill(text, width=92,
                        initial_indent=f'  {label:<26}', subsequent_indent=' ' * 28))
```

```
for section, what, why, how in CORRECTIONS:
    print(f'--- Розділ {section}')
    _block('що не так:', what)
    _block('чому неправильне:', why)
    _block('як правильно:', how)
    print()
```

--- Розділ 1
  що не так: Середнє від номінальної шкали: «помер/жив» містить мітки 1 і 2, а не величину
  чому неправильне: Цифри 1 і 2 — назви двох результатів лікування, а не їхня кількість. Жоден пацієнт не був «посередині між летальним і сприятливим результатом»: 61 помер, 60 вижило. 1.50 переказує лише те, що групи майже рівні; заміна кодів на 0/1 дала б 0.50, на 10/20 — 15.0, і дані при цьому не змінилися б жодного разу
  як правильно: Для номінальної шкали — тільки частоти й мода: померло 61 (50.4%), вижило 60 (49.6%). Якщо потрібне одне число, це летальність, тобто частка однієї категорії, а не середнє кодів

--- Розділ 2
  що не так: Частоти пораховано по сирому стовпцю: nunique() лічить написання, а не категорії
  чому неправильне: «Чотирьох статей» не існує — «ч» і «чол» це та сама категорія, записана двома способами, так само «ж» і «жін». Кожна з двох реальних категорій розірвана навпіл між своїми написаннями, тому й «найбільша група ж = 56 осіб (46%)» хибна: жінок насправді 75 (62%), чоловіків 46 (38%)
  як правильно: Спершу звести написання до категорій за першою літерою, і лише потім рахувати частоти. Будь-яка статистика категорій, порахована до зведення, описує охайність вводу, а не пацієнтів

--- Розділ 3
  що не так: Порядкова шкала: алфавітний порядок видано за порядок тяжкості, та ще й узято середнє
  чому неправильне: value_counts().sort_index() сортує рядки за абеткою, тому «найлегшим» став «важкий» (перший за алфавітом), а «найважчим» — «середній» (останній). На тих самих алфавітних кодах узято середнє 5.9, і воно вказало на «легкий». Жодне з трьох тверджень не стосується тяжкості: 12 рівнів тут — це 5 справжніх, розмножених друкарськими помилками й кінцевими пробілами
  як правильно: Звести 12 написань до 5 рівнів, задати порядок руками через pd.Categorical(ordered=True) і брати медіану, а не середнє: медіана = «середній», квартилі «середній»–«важкий»

--- Розділ 4
  що не так: Число обчислене на непридатних даних: у стовпці КТ два несумісні кодування
  чому неправильне: pd.to_numeric мовчки перетворив на NaN усі 17 текстових клітинок — 16 діапазонів («30-40%», «до 17%») і один рядок із самих пробілів, — тож «39.6%» описує 27 пацієнтів зі 121, тобто 22% відділення. Втрата не випадкова: саме в текстових клітинках лежать важчі випадки (середнє 47.5% проти 39.6% у числових), тому середнє ще й зміщене вниз. Розрив між середнім 39.6% і медіаною 30.0% має ту саму причину
  як правильно: Розібрати обидва кодування в один масштаб (частка × 100, діапазон — серединою). Тоді описано 43 пацієнти, середнє 42.5%, медіана 35.0%. Поруч обов’язково вказувати, що КТ описано лише у 44 зі 121 — число стосується цієї підвибірки, а не відділення

--- Розділ 5
  що не так: Число обчислене на непридатних даних: у spO2 змішано відсотки й частки
  чому неправильне: Мінімум 0.9 — це не сатурація 0.9%, несумісна з життям, а частка 0.9, тобто 90%, записана в іншому кодуванні. Одне таке значення тягне середнє вниз, і «82.05%» не описує ні справжню середню сатурацію, ні «виражену дихальну недостатність у більшості»: медіана 84% і тут чесніша за середнє, бо на поодиноке чуже значення не реагує
  як правильно: Знайти значення < 2 і домножити на 100 (або позначити пропуском), і лише потім рахувати. Межі фізіологічного діапазону (70–100%) мають перевірятися до будь-якої статистики

--- Розділ 6
  що не так: Коефіцієнт варіації від інтервальної шкали: у дат немає природного нуля
  чому неправильне: CV = σ/μ має зміст лише тоді, коли нуль шкали природний, інакше знаменник залежить від того, звідки ми почали рахувати. Тут μ — це середня відстань від 1 січня 1970 року, близько 19 000 днів, і вона не є властивістю госпіталізацій. Доказ: від епохи 1970 CV = 0.595%, від 1900 року — 0.25%, від 1 січня 2021 — 30.0%. Одні й ті самі дати, CV відрізняється у 120 разів, тож «надзвичайна однорідність» — властивість обраної точки відліку, а не потоку пацієнтів
  як правильно: Для інтервальної шкали CV не рахувати взагалі. Розсіяння дат описувати в одиницях різниці: розмах 463 дні (25.02.2021 – 03.06.2022), стандартне відхилення близько 113 днів — а це вже не однорідність, а півтора року роботи відділення

--- Розділ 7
  що не так: Пірсон між двома номінальними змінними, і висновок зроблено

	протилежний числу
чому неправильне:	r = 0.177 не можна читати як «лінійний зв’язок»: між двома наборами міток прямої не існує, а знак визначається тим, кого позначили нулем — заміна ч=0,ж=1 на ч=1,ж=0 дає −0.177 на тих самих даних. Але й за величиною висновок хибний: для таблиці 2×2 |r| збігається з V Крамера, і 0.177 — це не «відсутність зв’язку», а помітна різниця: летальність 57.3% у жінок проти 39.1% у чоловіків, тобто 18 в.п., χ^2 = 3.78 при одному ступені свободи (p ≈ 0.052)
як правильно:	Таблиця спряженості + χ^2 і V Крамера (V = 0.177), а для 2×2 ще інформативніше — різниця ризиків (18.2 в.п.) і відношення шансів (OR = 2.09). Ці міри не залежать від того, якими цифрами позначено категорії

--- Розділ 8

що не так:	Нормалізація застосована до всього, що має числовий тип, без перевірки варіативності й шкали
чому неправильне:	select_dtypes("number") дав 40 стовпців, серед яких ФЕРИТ2 має єдине непорожнє значення 1265: max == min, формула (x − min)/(max − min) дає 0/0 = NaN, і pandas мовчки лишає порожній стовпець замість помилки — у виводі describe() це видно як count = 0.0 і NaN у кожному рядку. Туди ж потрапив «помер/жив»: нормалізувати мітки 1 і 2 до 0 і 1 технічно можна, але отриманий стовпець не стає величиною. Теза «усі значення приведено до діапазону [0,1]» хибна буквально
як правильно:	Перед нормалізацією відсіяти стовпці з max == min і стовпці, числові лише за типом (коди, ідентифікатори). Нормалізувати поіменний список ознак шкали відношень, а не все, що int або float

--- Розділ 9

що не так:	Безумовний dropna() на наборі, де пропуск у частині стовпців структурний
чому неправильне:	Лишилось 0 рядків зі 121, і «повністю заповнений набір» — це порожня таблиця. Причина не в поганих даних: у ФЕРИТ2 заповнено 1 значення, у ФЕРИТ1 — 2, в ІЛ6-2 — 6, а «Дата розтину» порожня у всіх 60 вижившх за визначенням — там немає чого заповнювати. Жоден рядок не має усіх 77 значень, тому перетин завжди порожній
як правильно:	dropna() застосовувати лише до поіменного списку ознак, які справді підуть у модель, через subset=. Стовпці з часткою пропусків понад 50% виключати цілком, структурні пропуски кодувати окремою категорією або прапорцем, решту заповнювати медіаною

--- Розділ 10

що не так:	Витік цільової змінної: «Дата розтину» — наслідок смерті, а не її передвісник
чому неправильне:	r = 1.00 не означає сильного предиктора — він означає, що стовпець є самою ціллю в іншому записі: дата розтину заповнена рівно у 61 померлого і порожня рівно у 60 вижившх. До того ж кореляція рахується не з дат: .astype("int64") перетворив NaT на −9.22·10^18, тож фактично корелюють «тут стоїть службове число» з «пацієнт вижив». Модель на таких ознаках дасть 100% на навчальній вибірці й буде непридатна для прогнозу — на момент прогнозу розтину ще немає. Решту рейтингу теж не можна зіставляти: кожне r пораховане на своїй підмножині пацієнтів
як правильно:	Викинути з кандидатів усе, що фізично виникає після результату (Дата розтину, Виписка, ЛН, Ліжко днів). Для бінарної цілі брати точковий бісеріальний коефіцієнт або AUC, рахувати їх на спільній підмножині й поруч указувати n, на якому пораховано кожен показник

Виправте щонайменше три помилки в коді

Оберіть три з десяти та покажіть правильне обчислення поруч із неправильним.

In [26]:
```
# ваш код: три правильні обчислення замість хибних
# Беру три з десяти — ті, де різниця між «неправильно» і «правильно» вимірюється числом.
LINE = '—' * 78

# == РОЗДІЛ 3 == алфавіт замість порядку тяжкості + середнє замість медіани
print(LINE); print('РОЗДІЛ 3. Тяжкість ураження легень'); print(LINE)

degree_raw = df['ступінь КТ'].value_counts().sort_index()
alpha = {lvl: i for i, lvl in enumerate(degree_raw.index)}
mean_alpha = df['ступінь КТ'].map(alpha).mean()
print(f'БУЛО : рівнів {len(degree_raw)}, «найлегший» = {degree_raw.index[0]!r}, '
      f'«найважчий» = {degree_raw.index[-1]!r}')
print(f'       середній ступінь = {mean_alpha:.2f} -> {degree_raw.index[int(round(mean_alpha))]!r}')
```

```
degree = CLEAN['ступінь КТ']
deg_codes = pd.Series(pd.Categorical(degree.dropna(), categories=CT_ORDER, ordered=True).codes)
print(f'СТАЛО: рівнів {degree.nunique()}, найлегший = {CT_ORDER[0]!r}, найважчий = {CT_ORDER[-1]!r}')
print(f'       медіана = {CT_ORDER[int(np.floor(deg_codes.median()))]!r}, '
      f'квартилі = {[CT_ORDER[int(np.floor(q))] for q in deg_codes.quantile([0.25, 0.75])]}')
print(f'       частоти: {degree.value_counts().reindex(CT_ORDER).to_dict()}')
print()

# == РОЗДІЛ 4 == два кодування в одному стовпці
print(LINE); print('РОЗДІЛ 4. Обсяг ураження легень'); print(LINE)

ct_raw = pd.to_numeric(df['КТ'], errors='coerce')
ct_fixed = CLEAN['КТ']
print(f'БУЛО : середнє {100*ct_raw.mean():.1f}%, медіана {100*ct_raw.median():.1f}%  '
      f'— з {int(ct_raw.notna().sum())} пацієнтів зі {len(df)}')
print(f'СТАЛО: середнє {ct_fixed.mean():.1f}%, медіана {ct_fixed.median():.1f}%  '
      f'— з {int(ct_fixed.notna().sum())} пацієнтів зі {len(df)}')
print(f'       КТ взагалі не описано у {int(df["КТ"].isna().sum())} пацієнтів — це пишемо поруч із числом')
print()

# == РОЗДІЛ 2 == частоти пораховано по сирому стовпцю
print(LINE); print('РОЗДІЛ 2. Розподіл пацієнтів за статтю'); print(LINE)

sex_raw = df['Стать'].value_counts()
n_sex = int(df['Стать'].notna().sum())
print(f'БУЛО : «статей» {df["Стать"].nunique()} — {sex_raw.to_dict()}')
print(f'       найбільша група {sex_raw.index[0]!r} = {sex_raw.iloc[0]} осіб '
      f'({100 * sex_raw.iloc[0] / n_sex:.0f}%)')

sex_fixed = CLEAN['Стать'].value_counts()
print(f'СТАЛО: категорій {CLEAN["Стать"].nunique()} — {sex_fixed.to_dict()}')
print(f'       найбільша група {sex_fixed.index[0]!r} = {sex_fixed.iloc[0]} осіб '
      f'({100 * sex_fixed.iloc[0] / n_sex:.0f}%)')
print(f'       «ч» і «чол» — та сама категорія, записана двома способами; так само «ж» і «жін». '
      f'Кожна з двох реальних')
print(f'       категорій була розірвана навпіл між написаннями, тому й частка '
      f'{100 * sex_raw.iloc[0] / n_sex:.0f}% занижена на '
      f'{100 * (sex_fixed.iloc[0] - sex_raw.iloc[0]) / n_sex:.0f} в.п.')
print()
```

```
──────────────────────────────────────────────────────────────────────
РОЗДІЛ 3. Тяжкість ураження легень
──────────────────────────────────────────────────────────────────────
БУЛО : рівнів 12, «найлегший» = 'важкий', «найважчий» = 'середній'
       середній ступінь = 5.90 -> 'легкий'
СТАЛО: рівнів 5, найлегший = 'легкий', найважчий = 'критичний'
       медіана = 'середній', квартилі = ['середній', 'важкий']
       частоти: {'легкий': 7, 'середній': 18, 'важкий': 10, 'вкрай важкий': 4, 'критичний': 3}

──────────────────────────────────────────────────────────────────────
РОЗДІЛ 4. Обсяг ураження легень
──────────────────────────────────────────────────────────────────────
БУЛО : середнє 39.6%, медіана 30.0%  — з 27 пацієнтів зі 121
СТАЛО: середнє 42.5%, медіана 35.0%  — з 43 пацієнтів зі 121
       КТ взагалі не описано у 77 пацієнтів — це пишемо поруч із числом

──────────────────────────────────────────────────────────────────────
РОЗДІЛ 2. Розподіл пацієнтів за статтю
──────────────────────────────────────────────────────────────────────
БУЛО : «статей» 4 — {'ж': 56, 'ч': 34, 'жін': 19, 'чол': 12}
       найбільша група 'ж' = 56 осіб (46%)
СТАЛО: категорій 2 — {'ж': 75, 'ч': 46}
       найбільша група 'ж' = 75 осіб (62%)
       «ч» і «чол» — та сама категорія, записана двома способами; так само «ж» і «жін». Кожна з двох реальних
       категорій була розірвана навпіл між написаннями, тому й частка 46% занижена на 16 в.п.
```

Етап 7. Висновки та індивідуальне питання

In [27]:

```
CONCLUSIONS = [
    # три висновки, кожен — повне речення про ваші дані, а не загальна фраза

    'Чотири з восьми моїх стовпців містять по два несумісні кодування одночасно, і наївне '
    'pd.to_numeric мовчки викидає менш зручне з них: у «КТ» воно лишає 27 значень із 44, '
    'у «Т вище 37,5» — 30 із 78, у «Телефон» — 23 зі 120; при цьому втрата не випадкова, бо '
    'текстові клітинки «КТ» описують у середньому 47.5% ураження проти 39.6% у числових, тож '
    'наївне середнє не просто неповне, а систематично занижене.',
```

```
    'Для пари «Стать × ступінь КТ» вибір міри змінює не точність, а сам статус результату: шість '
    'різних кодувань тих самих 42 пацієнтів дали кореляцію Пірсона від −0.1303 до +0.1303, тоді як '
    'коефіцієнт Крамера в усіх шести випадках дорівнював 0.2994, бо рахується з частот у таблиці '
    'спряженості, а не зі значень кодів — тобто r виміряв мій довільний dict, а V виміряв дані.',

    'Самі по собі 42 спостереження із заповненим «ступінь КТ» не є випадковою підвибіркою зі 121: '
    'серед них вижило 81% проти 33% серед решти, а опис КТ мають 46% чоловіків проти 28% жінок, '
    'тому V = 0.299 коректно описує лише тих, хто дожив до опису знімка, і переносити це число на '
    'все відділення не можна — обмеженням тут є вибірка, а не міра.',
]
assert len(CONCLUSIONS) >= 3 and all(len(concl) >= 60 for concl in CONCLUSIONS), \
    'Потрібні три змістовні висновки'
for i, concl in enumerate(CONCLUSIONS, 1):
    print(f'{i}. {concl}')
    print()
```

1. Чотири з восьми моїх стовпців містять по два несумісні кодування одночасно, і наївне pd.to_numeric мовчки викидає менш зручне з них: у «КТ» воно лишає 27 значень із 44, у «Т вище 37,5» — 30 із 78, у «Телефон» — 23 зі 120; при цьом у втрата не випадкова, бо текстові клітинки «КТ» описують у середньому 47.5% ураження проти 39.6% у числових, тож на ївне середнє не просто неповне, а систематично занижене.

2. Для пари «Стать × ступінь КТ» вибір міри змінює не точність, а сам статус результату: шість різних кодувань тих с амих 42 пацієнтів дали кореляцію Пірсона від −0.1303 до +0.1303, тоді як коефіцієнт Крамера в усіх шести випадках до рівнював 0.2994, бо рахується з частот у таблиці спряженості, а не зі значень кодів — тобто r виміряв мій довільний dict, а V виміряв дані.

3. Самі по собі 42 спостереження із заповненим «ступінь КТ» не є випадковою підвибіркою зі 121: серед них вижило 81% проти 33% серед решти, а опис КТ мають 46% чоловіків проти 28% жінок, тому V = 0.299 коректно описує лише тих, хто д ожив до опису знімка, і переносити це число на все відділення не можна — обмеженням тут є вибірка, а не міра.

In [28]:

```
# Числа, на які спираються три висновки — щоб їх можна було перевірити, а не вірити на слово

print('Висновок 1 — скільки значень втрачає наївний парсер і чи випадкова втрата:')
loss = pd.DataFrame(
    [(c,
      int(df[c].notna().sum()),
      int(pd.to_numeric(df[c], errors='coerce').notna().sum()))
     for c in ['КТ', 'Т вище 37,5', 'Телефон']],
    columns=['стовпець', 'непорожніх', 'наївно']).set_index('стовпець')
loss['втрачено'] = loss['непорожніх'] - loss['наївно']
print(loss.to_string())

is_number = df['КТ'].map(lambda v: isinstance(v, (int, float, np.integer, np.floating)) and pd.notna(v))
is_text = df['КТ'].map(lambda v: isinstance(v, str) and v.strip() != '')
print(f'       «КТ»: текстові клітинки {CLEAN["КТ"][is_text].mean():.1f}% проти '
      f'{CLEAN["КТ"][is_number].mean():.1f}% у числових — втрата не випадкова')
print()

print('Висновок 2 — r залежить від кодування, V не залежить:')
r_list, V_list = [], []
for sex_code in sex_codings.values():
    for degree_code in degree_codings.values():
        r_list.append(sex_c.map(sex_code).corr(degree_c.map(degree_code)))
        V_list.append(cramer_v(pd.crosstab(sex_c.map(sex_code),
                                           degree_c.map(degree_code)))[1])
print(f'       r Пірсона: {len(set(np.round(r_list, 4)))} різних значень '
      f'від {min(r_list):+.4f} до {max(r_list):+.4f}')
print(f'       V Крамера: {len(set(np.round(V_list, 4)))} значення — {V_list[0]:.4f} в усіх '
      f'{len(V_list)} кодуваннях')
print()

print('Висновок 3 — хто потрапив у підвибірку з описаним ступенем (% у рядку):')
has_degree = CLEAN['ступінь КТ'].notna().map({True: 'ступінь є', False: 'ступеня немає'})
print(pd.crosstab(has_degree, df['помер/жив'].map({1: 'помер', 2: 'вижив'}),
                  normalize='index').mul(100).round(1).to_string())
print()
print(pd.crosstab(CLEAN['Стать'], has_degree, normalize='index').mul(100).round(1).to_string())
```

```
Висновок 1 — скільки значень втрачає наївний парсер і чи випадкова втрата:
             непорожніх  наївно  втрачено
стовпець
КТ                   44      27        17
Т вище 37,5          78      30        48
Телефон             120      23        97
       «КТ»: текстові клітинки 47.5% проти 39.6% у числових — втрата не випадкова

Висновок 2 — r залежить від кодування, V не залежить:
       r Пірсона: 4 різних значень від -0.1303 до +0.1303
       V Крамера: 1 значення — 0.2994 в усіх 6 кодуваннях

Висновок 3 — хто потрапив у підвибірку з описаним ступенем (% у рядку):
```

```
помер/жив       вижив  помер
ступінь КТ
ступеня немає   32.9   67.1
ступінь є       81.0   19.0

ступінь КТ  ступеня немає  ступінь є
Стать
ж                    72.0       28.0
ч                    54.3       45.7
```

In [29]:

```
print('ВАШЕ ПИТАННЯ:'); print(VARIANT['питання']); print()

ANSWER = '''
КОРОТКА ANSWER. Пірсон не можна, бо обидві змінні номінальні: він будується з середніх
і стандартних відхилень САМИХ ЗНАЧЕНЬ, а значення тут — мітки, а не величини. Брати слід
χ² з коефіцієнтом Крамера, а для таблиці 2×2 ще й різницю ризиків із відношенням шансів.

1. ЧОМУ ПІРСОН НЕДОПУСТИМИЙ

r = cov(X,Y) / (σx·σy) вимагає, щоб над значеннями X і Y були визначені додавання,
віднімання й ділення на кількість. Для шкали відношень це так: 70 і 80 ударів пульсу
можна відняти. Для статі — ні: «ч» і «ж» це назви груп, і «ч + ж», поділене на 2, не
позначає нічого. Те саме з «помер/жив»: 1 і 2 — номери категорій, а не кількості. Те,
що в Excel вони записані цифрами, робить обчислення технічно можливим, а не осмисленим.

Три конкретні наслідки, кожен видно на цих даних:

а) ЗНАК ДОВІЛЬНИЙ. Кодування ч=0, ж=1 дає r = +0.177, кодування ч=1, ж=0 дає r = −0.177.
   Дані ті самі — змінився лише мій dict. Величина, напрямок якої я обираю сам, не може
   описувати напрямок зв'язку в даних.

б) «ЛІНІЙНОГО ЗВ'ЯЗКУ» НЕ ІСНУЄ. Пірсон міряє, наскільки хмара точок лягає на пряму.
   Коли в кожній змінній рівно два значення, точок усього чотири — по одній на клітинку
   таблиці 2×2, — і через них завжди можна провести пряму. Питання про її нахил порожнє.

в) ВЕЛИЧИНА ЗАЛЕЖИТЬ ВІД КОДУВАННЯ, щойно категорій більше двох. На парі мого варіанта
   «Стать × ступінь КТ» перехід від кодів за тяжкістю до кодів за алфавітом змінює r
   з 0.1303 на 0.0144 — удев'ятеро, на тих самих 42 пацієнтах (таблиця на етапі 4).

2. НЮАНС, ЯКИЙ ПСУЄ ПРОСТУ ANSWER

Саме для таблиці 2×2 |r| збігається з коефіцієнтом φ, а φ для 2×2 дорівнює V Крамера.
Я перевірив це на цих даних: |r| = 0.1767 і V = 0.1767 до четвертого знака. Тобто ЧИСЛО
в розділі 7 помилкового звіту не є сміттям — сміттям є те, як його прочитали.

Точне формулювання таке: на двох бінарних змінних r припустимо використовувати як міру
СИЛИ зв'язку, усвідомлюючи, що це φ, і читаючи тільки |r|. Недопустимо читати його як
кореляцію: ні знак, ні слово «лінійний» змісту не мають. А щойно хоч одна змінна має три
категорії й більше, зникає й ця поблажка — там r стає функцією від нумерації.

3. ЩО БРАТИ ЗАМІСТЬ

Таблиця спряженості, χ² і V Крамера:

      помер/жив    вижив   помер    летальність
      ж               32      43         57.3 %
      ч               28      18         39.1 %
      χ² = 3.779 (1 ст. свободи),  V = 0.177,  мінімальна очікувана частота 22.8

Для 2×2 інформативніші ще дві міри, бо мають клінічний зміст:
      різниця ризиків   = 57.3 − 39.1 = 18.2 відсоткових пункти;
      відношення шансів = (43/32)/(18/28) = 2.09 — шанси померти у жінок удвічі вищі.

Жодна з трьох не залежить від того, якими цифрами позначено категорії: V, як показано
на етапі 4, не змінюється від перестановки кодів зовсім, а OR при перестановці переходить
у 1/OR, тобто зберігає зміст.

4. І ВИСНОВОК ЗВІТУ ХИБНИЙ НАВІТЬ ЗА ЙОГО ВЛАСНИМ ЧИСЛОМ

«0.177 близьке до нуля, зв'язку немає» — хибно двічі. По-перше, 0.177 для таблиці 2×2 це
V Крамера, а не «майже нуль»: за ним стоїть різниця летальності у 18 відсоткових пунктів.
По-друге, χ² = 3.78 при одному ступені свободи відповідає p ≈ 0.052 — це межа звичайного
порога, тобто «не довели» і аж ніяк не «показали відсутність». Відсутність зв'язку взагалі
не доводиться незначущим результатом, тим паче на вибірці зі 121 пацієнта.
'''

assert len(ANSWER.strip()) >= 200, 'Відповідь має бути розгорнутою (від 200 символів)'
print(ANSWER)
```

ВАШЕ ПИТАННЯ:
Чому не можна обчислити коефіцієнт кореляції Пірсона між статтю та результатом лікування? Яку міру зв'язку слід узят
и замість нього?

КОРОТКА ANSWER. Пірсон не можна, бо обидві змінні номінальні: він будується з середніх
і стандартних відхилень САМИХ ЗНАЧЕНЬ, а значення тут — мітки, а не величини. Брати слід
χ^2 з коефіцієнтом Крамера, а для таблиці 2×2 ще й різницю ризиків із відношенням шансів.

1. ЧОМУ ПІРСОН НЕДОПУСТИМИЙ

r = cov(X,Y) / (σx·σy) вимагає, щоб над значеннями X і Y були визначені додавання,
віднімання й ділення на кількість. Для шкали відношень це так: 70 і 80 ударів пульсу
можна відняти. Для статі — ні: «ч» і «ж» це назви груп, і «ч + ж», поділене на 2, не
позначає нічого. Те саме з «помер/жив»: 1 і 2 — номери категорій, а не кількості. Те,
що в Excel вони записані цифрами, робить обчислення технічно можливим, а не осмисленим.

Три конкретні наслідки, кожен видно на цих даних:

а) ЗНАК ДОВІЛЬНИЙ. Кодування ч=0, ж=1 дає r = +0.177, кодування ч=1, ж=0 дає r = −0.177.
   Дані ті самі — змінився лише мій dict. Величина, напрямок якої я обираю сам, не може
   описувати напрямок зв'язку в даних.

б) «ЛІНІЙНОГО ЗВ'ЯЗКУ» НЕ ІСНУЄ. Пірсон міряє, наскільки хмара точок лягає на пряму.
   Коли в кожній змінній рівно два значення, точок усього чотири — по одній на клітинку
   таблиці 2×2, — і через них завжди можна провести пряму. Питання про її нахил порожнє.

в) ВЕЛИЧИНА ЗАЛЕЖИТЬ ВІД КОДУВАННЯ, щойно категорій більше двох. На парі мого варіанта
   «Стать × ступінь КТ» перехід від кодів за тяжкістю до кодів за алфавітом змінює r
   з 0.1303 на 0.0144 — удев'ятеро, на тих самих 42 пацієнтах (таблиця на етапі 4).

2. НЮАНС, ЯКИЙ ПСУЄ ПРОСТУ ANSWER

Саме для таблиці 2×2 |r| збігається з коефіцієнтом φ, а φ для 2×2 дорівнює V Крамера.
Я перевірив це на цих даних: |r| = 0.1767 і V = 0.1767 до четвертого знака. Тобто ЧИСЛО
в розділі 7 помилкового звіту не є сміттям — сміттям є те, як його прочитали.

Точне формулювання таке: на двох бінарних змінних r припустимо використовувати як міру
СИЛИ зв'язку, усвідомлюючи, що це φ, і читаючи тільки |r|. Недопустимо читати його як
кореляцію: ні знак, ні слово «лінійний» змісту не мають. А щойно хоч одна змінна має три
категорії й більше, зникає й ця поблажка — там r стає функцією від нумерації.

3. ЩО БРАТИ ЗАМІСТЬ

Таблиця спряженості, χ^2 і V Крамера:

помер/жив	вижив	помер	летальність
ж	32	43	57.3 %
ч	28	18	39.1 %

χ^2 = 3.779 (1 ст. свободи), V = 0.177, мінімальна очікувана частота 22.8

Для 2×2 інформативніші ще дві міри, бо мають клінічний зміст:
      різниця ризиків = 57.3 − 39.1 = 18.2 відсоткових пункти;
      відношення шансів = (43/32)/(18/28) = 2.09 — шанси померти у жінок удвічі вищі.

Жодна з трьох не залежить від того, якими цифрами позначено категорії: V, як показано
на етапі 4, не змінюється від перестановки кодів зовсім, а OR при перестановці переходить
у 1/OR, тобто зберігає зміст.

4. І ВИСНОВОК ЗВІТУ ХИБНИЙ НАВІТЬ ЗА ЙОГО ВЛАСНИМ ЧИСЛОМ

«0.177 близьке до нуля, зв'язку немає» — хибно двічі. По-перше, 0.177 для таблиці 2×2 це
V Крамера, а не «майже нуль»: за ним стоїть різниця летальності у 18 відсоткових пунктів.
По-друге, χ^2 = 3.78 при одному ступені свободи відповідає p ≈ 0.052 — це межа звичайного
порога, тобто «не довели» і аж ніяк не «показали відсутність». Відсутність зв'язку взагалі
не доводиться незначущим результатом, тим паче на вибірці зі 121 пацієнта.

In [30]:
```
# Числова перевірка всього, що стверджує відповідь
died_flag = (df['помер/жив'] == 1).astype(int)
r_direct = sex_c.map({'ч': 0, 'ж': 1}).corr(died_flag)
r_reversed = sex_c.map({'ч': 1, 'ж': 0}).corr(died_flag)
tab = pd.crosstab(sex_c, df['помер/жив'].map({1: 'помер', 2: 'вижив'}))
chi2_, V_, n_, exp_ = cramer_v(tab)
lethality_ = tab['помер'] / tab.sum(axis=1) * 100
OR_ = (tab.loc['ж', 'помер'] / tab.loc['ж', 'вижив']) / (tab.loc['ч', 'помер'] / tab.loc['ч', 'вижив'])

print(f'r (ч=0, ж=1)         = {r_direct:+.4f}')
print(f'r (ч=1, ж=0)         = {r_reversed:+.4f}   <- ті самі дані, інший знак')
print(f'|r|                  = {abs(r_direct):.4f}')
print(f'V Крамера            = {V_:.4f}   <- збігається з |r|: {bool(np.isclose(abs(r_direct), V_))}')
print()
print(f'летальність ж        = {lethality_["ж"]:.1f} %')
print(f'летальність ч        = {lethality_["ч"]:.1f} %')
```

```
print(f'різниця ризиків      = {lethality_["ж"] - lethality_["ч"]:.1f} в.п.')
print(f'відношення шансів OR = {OR_:.2f}')
print(f'χ²                   = {chi2_:.4f} (1 ст. свободи)')

# p-значення для одного ступеня свободи: χ² з 1 ст. св. — це квадрат стандартної нормальної
# величини, тому P(χ² > x) = 2·(1 − Φ(√x)) = erfc(√(x/2)). Без scipy, зі стандартного math.
p = math.erfc(math.sqrt(chi2_ / 2))
print(f'p-значення           = {p:.4f}   <- межа порога 0.05, а не «зв'язку немає»')
print()
print(f'і те саме для пари мого варіанта: V = {V:.4f} незмінний за будь-якого кодування,')
print('тоді як r гуляє від -0.1303 до +0.1303 — див. таблицю на етапі 4')
```

```
r (ч=0, ж=1)         = +0.1767
r (ч=1, ж=0)         = -0.1767   <- ті самі дані, інший знак
|r|                  = 0.1767
V Крамера            = 0.1767   <- збігається з |r|: True

летальність ж        = 57.3 %
летальність ч        = 39.1 %
різниця ризиків      = 18.2 в.п.
відношення шансів OR = 2.09
χ²                   = 3.7792 (1 ст. свободи)
p-значення           = 0.0519   <- межа порога 0.05, а не «зв'язку немає»

і те саме для пари мого варіанта: V = 0.2994 незмінний за будь-якого кодування,
тоді як r гуляє від -0.1303 до +0.1303 — див. таблицю на етапі 4
```

Етап 8. Декларація про використання інструментів ШІ

Користуватися мовними моделями дозволено. Це нормальна інженерна практика.

Обов'язковою є прозорість: ви вказуєте, де саме і як скористалися інструментом — так само, як у науковій статті вказують використане програмне забезпечення. Задекларована допомога порушенням не є.

Порушенням є інше: здати роботу, яку ви не можете пояснити. На захисті вас попросять показати конкретний рядок свого коду, пояснити, чому він саме такий, і внести в нього невелику зміну на місці.

Для кожного етапу оберіть одне з формулювань:

- `'ні'` — інструментами ШІ не користувався
- `'синтаксис'` — питав про назву функції або синтаксис, код писав сам
- `'код, перевірено'` — код згенеровано, я його прочитав, перевірив і розумію
- `'текст, перероблено'` — згенеровано чернетку тексту, яку я переписав по суті
- `'текст, як є'` — згенерований текст залишено майже без змін

In [31]:
```
DECLARATION = {
    'етап 1 — паспорт змінних':        'текст, перероблено',
    'етап 2 — допустимі статистики':   'код, перевірено',
    'етап 3 — безглузді числа':        'код, перевірено',
    'етап 4 — міра зв'язку':           'код, перевірено',
    'етап 5 — підготовка до моделі':   'код, перевірено',
    'етап 6 — виправлення аналізу':    'текст, перероблено',
    'етап 7 — висновки й відповідь':   'текст, перероблено',
}

WHAT_I_CHECKED = (
    'Перерахував кожне число, на яке спираюся. Звірив 12 написань «ступінь КТ» з рівнями CT_ORDER '
    'і переконався, що clean_degree не губить жодного запису (42 з 42) і не плутає «сердній» із '
    '«середній». Перевірив clean_ct на всіх 17 текстових клітинках: «30-40%» дає 35.0, «до 17%» дає '
    '17.0, рядок із самих пробілів — NaN; разом розібрано 43 клітинки з 44, тоді як pd.to_numeric '
    'розбирає 27. Окремо впевнився, що текстові клітинки систематично важчі (47.5% проти 39.6%), '
    'тож втрата не випадкова. Для етапу 4 перерахував χ² вручну з таблиці спряженості за формулою '
    'Σ(сп−оч)²/оч і звірив із V = 0.2994; перевірив інваріантність V перестановкою кодів у шести '
```

```
    'варіантах і бачив, що він не змінюється, а r змінюється. Для етапу 7 перевірив рівність |r| = V '
    'на таблиці 2×2 (0.1767 обидва) і перерахував OR = 2.09 вручну з чотирьох частот. Для етапу 6 '
    'відтворив кожне з десяти хибних чисел звіту своїм кодом перед тим, як пояснювати їх: зокрема '
    'перевірив, що CV дат змінюється з 0.595% на 30.0% при зміні точки відліку, і що NaT після '
    'astype("int64") стає −9.22·10^18. Прогнав ноутбук через Restart & Run All на чистому ядрі.'
)
I_CONFIRM = True      # я можу пояснити кожен рядок свого коду
                        # і внести в нього зміни на вимогу під час захисту
```

In [32]:

```
# --- ФІНАЛЬНА САМОПЕРЕВІРКА ---
WORDINGS = {'ні', 'синтаксис', 'код, перевірено', 'текст, перероблено', 'текст, як є'}

checks = {
    'заповнено титульні поля':        bool(FULL_NAME and GROUP and NUMBER),
    'етап 1: паспорт із 8 стовпців':  all(PASSPORT[c][0] and PASSPORT[c][1] for c in MY_COLS),
    'етап 3: три безглузді числа':    len(MEANINGLESS) >= 3,
    'етап 4: міру обрано й пояснено': bool(CHOSEN_MEASURE and len(WHY_THIS_ONE) >= 40),
    'етап 5: підготовку описано':     all(len(PREPARATION[c].strip()) >= 15 for c in MY_COLS),
    'етап 6: знайдено 10 помилок':    len(CORRECTIONS) == 10,
    'етап 7: три висновки':           len(CONCLUSIONS) >= 3,
    'етап 7: відповідь на питання':   len(ANSWER.strip()) >= 200,
    'декларацію заповнено':           all(v in WORDINGS for v in DECLARATION.values()),
    'опис перевірки ≥ 120 символів':  len(WHAT_I_CHECKED.strip()) >= 120,
    'підтвердження авторства':        I_CONFIRM is True,
}

print('ЧЕК-ЛИСТ ГОТОВНОСТІ')
print('=' * 52)
for name, ok in checks.items():
    print(('  ГОТОВО     ' if ok else '  НЕ ГОТОВО ') + name)
print('=' * 52)

assert all(checks.values()), 'Робота не готова до здачі — див. чек-лист вище'
print()
print(f'Робота варіанта {VARIANT["варіант"]} готова до захисту.')
print(f'{FULL_NAME}, {GROUP}')
```

```
ЧЕК-ЛИСТ ГОТОВНОСТІ
====================================================
  ГОТОВО     заповнено титульні поля
  ГОТОВО     етап 1: паспорт із 8 стовпців
  ГОТОВО     етап 3: три безглузді числа
  ГОТОВО     етап 4: міру обрано й пояснено
  ГОТОВО     етап 5: підготовку описано
  ГОТОВО     етап 6: знайдено 10 помилок
  ГОТОВО     етап 7: три висновки
  ГОТОВО     етап 7: відповідь на питання
  ГОТОВО     декларацію заповнено
  ГОТОВО     опис перевірки ≥ 120 символів
  ГОТОВО     підтвердження авторства
====================================================

Робота варіанта 15 готова до захисту.
Цимбалюк Антон, ШІ-34
```

Перед здачею

Виконайте **Kernel → Restart & Run All**. Ноутбук має пройти згори донизу без жодної помилки — це перше, що перевіряється на захисті.